

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG HCM

KHOA TOÁN - TIN HỌC



TIỂU LUẬN MÔN  
QUY HOẠCH TUYẾN TÍNH

**ĐỀ TÀI**

VIẾT CHƯƠNG TRÌNH GIẢI BÀI TOÁN QUY  
HOẠCH TUYẾN TÍNH

GV hướng dẫn: PSG.TS. Nguyễn Lê Hoàng Anh

Nhóm sinh viên thực hiện:

| Họ và tên              | MSSV     | Mã lớp |
|------------------------|----------|--------|
| Trần Minh Trực         | 22110241 | 23TTH  |
| Huỳnh Thị Cẩm Tú       | 19110498 | 23TTH  |
| Vũ Ngọc Quỳnh Chi      | 21110254 | 23TTH  |
| Lê Thái Minh Tâm       | 21110173 | 23TTH  |
| Nguyễn Đình Nhật Cường | 22110030 | 23TTH  |

Thành phố Hồ Chí Minh, 2025

## BẢNG PHÂN CÔNG NHIỆM VỤ

| Họ Và Tên                    | MSSV     | Bảng Phân Công Nhiệm Vụ                |
|------------------------------|----------|----------------------------------------|
| Trần Minh Trực (nhóm trưởng) | 22110241 | Lập bảng phân công nhiệm vụ + đánh giá |
| Huỳnh Thị Cẩm Tú             | 19110498 | Hỗ trợ gõ Latex                        |
| Vũ Ngọc Quỳnh Chi            | 21110254 | Tóm tắt lý thuyết các dạng QHTT        |
| Lê Thái Minh Tâm             | 21110173 | Tóm tắt lý thuyết các dạng QHTT        |
| Nguyễn Đình Nhật Cường       | 22110030 | Code bài toán QHTT                     |

# 1 Nội dung

## 1.1 Giải bài toán quy hoạch tuyến tính hai biến bằng phương pháp hình học

Các bước làm như sau:

- **Bước 1:** Xác định hàm mục tiêu  $z$  và thay tọa độ điểm của bài toán quy hoạch tuyến tính.

|              | $O$      | $A$ | $B$ | $C$ | $\dots$ |
|--------------|----------|-----|-----|-----|---------|
| $(x_1, x_2)$ | $(0, 0)$ |     |     |     |         |
| $z$          | 0        |     |     |     |         |

- **Bước 2:** Vẽ hình và miền nghiệm.
- **Bước 3:** Kết luận nghiệm tối ưu và giá trị tối ưu

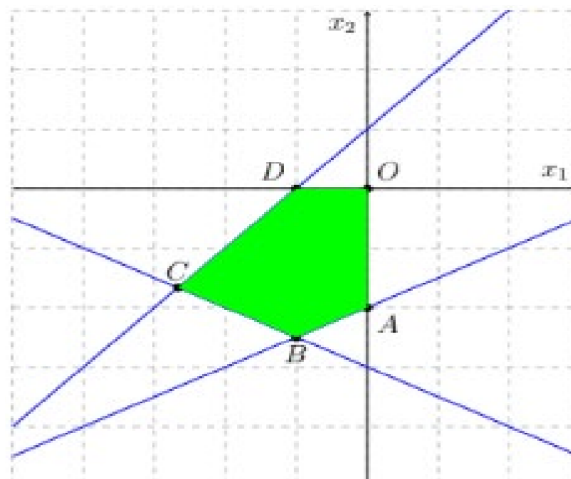
**Ví dụ:** Dùng phương pháp hình học giải bài toán quy hoạch tuyến tính sau

$$\begin{aligned} \min z &= -x_1 + x_2 \\ (P) : \begin{cases} -x_1 - 2x_2 \leq 6 \\ x_1 - 2x_2 \leq 4 \\ -x_1 + x_2 \leq 1 \\ x_1 \leq 0 \\ x_2 \leq 0 \end{cases} \end{aligned}$$

Thay tọa độ điểm:

|              | $O$      | $A$       | $B$                             | $C$                                        | $D$        |
|--------------|----------|-----------|---------------------------------|--------------------------------------------|------------|
| $(x_1, x_2)$ | $(0, 0)$ | $(0, -2)$ | $\left(-1, -\frac{5}{2}\right)$ | $C\left(-\frac{8}{3}, -\frac{5}{3}\right)$ | $D(-1, 0)$ |
| $z$          | 0        | -2        | $-\frac{3}{2}$                  | 1                                          | 1          |

Miền nghiệm của  $(P)$  có dạng là:



Vậy: nghiệm tối ưu của  $(P)$  là  $(x_1, x_2) = (0, -2)$  và giá trị tối ưu của  $(P)$  là  $z = -2$ .

## 1.2 Giải bài toán quy hoạch tuyến tính bằng phương pháp đơn hình

- **Bước 1:** Xây dựng từ vệt xuất phát.

– Cho biến không cơ sở  $= 0$  ( $G > 0$ )  $\rightarrow$  Đọc giá trị biến cơ sở.

$z = c_1x_1 + c_2x_2 + \dots + c_nx_n \rightarrow$  biến không cơ sở.

$w_1 = b_1 - a_{11}x_1 - a_{12}x_2 - a_{1n}x_n.$

$w_2 = b_2 - a_{21}x_1 - a_{22}x_2 - a_{2n}x_n.$

...

$w_n = b_n - a_{n1}x_1 - a_{n2}x_2 - a_{nn}x_n.$

- **Bước 2:** Thực hiện chọn biến vào/ra.

– Chọn biến vào cơ sở: chọn biến (không cơ sở) có hệ số  $G$  âm nhất ( $x_j$ ) (cột ứng với  $x_j$ ).

– Chọn biến ra cơ sở: chỉ lưu ý cột  $x_j$ . Trên cột  $x_j$ , chỉ lấy những dòng mà hệ số trước đó âm.

- **Bước 3:** Tiến hành xoay từ vệt  $\rightarrow$  Từ vệt mới.

- **Bước 4:** Kiểm tra điều kiện dừng. Với mọi  $G \geq 0 \rightarrow$  dừng.

Từ vệt lớn nhất là từ vệt tối ưu  $\rightarrow$  đọc nghiệm và giá trị tối ưu của  $(P')$   $\rightarrow$  trả lại nghiệm và giá trị tối ưu của  $(P)$ .

**Ví dụ:**

Đặt  $y_1 = -x_1$ ,  $y_2 = -x_2$ . Dạng chuẩn của  $(P)$ :

$$(P)' \left\{ \begin{array}{l} \min \quad y_1 - y_2 \\ y_1 + 2y_2 \leq 6 \\ -y_1 + 2y_2 \leq 4 \\ y_1 - y_2 \leq 1 \\ y_1 \geq 0 \\ y_2 \geq 0 \end{array} \right.$$

|       |     |       |        |         |      |       |      |                   |                   |                   |
|-------|-----|-------|--------|---------|------|-------|------|-------------------|-------------------|-------------------|
| $z'$  | $=$ | $y_1$ | $-y_2$ |         | $z'$ | $=$   | $-2$ | $+\frac{1}{2}y_1$ | $+\frac{1}{2}w_2$ |                   |
| $w_1$ | $=$ | $6$   | $-y_1$ | $-2y_2$ |      | $w_1$ | $=$  | $2$               | $-2y_1$           | $+w_2$            |
| $w_2$ | $=$ | $4$   | $+y_1$ | $-2y_2$ |      | $y_2$ | $=$  | $2$               | $+\frac{1}{2}y_1$ | $-\frac{1}{2}w_2$ |
| $w_3$ | $=$ | $1$   | $-y_1$ | $+y_2$  |      | $w_3$ | $=$  | $3$               | $-\frac{1}{2}y_1$ | $-\frac{1}{2}w_2$ |

$$\text{Chọn } y_1 = w_2 = 0 \Rightarrow \left\{ \begin{array}{l} w_1 = 2 \\ y_2 = 2 \\ w_3 = 3 \end{array} \right.$$

Vậy bài toán  $(P)$  có nghiệm tối ưu là  $x_1 = 0$ ,  $x_2 = -2$  và giá trị tối ưu là  $z = -2$ .



- Chọn biến vào: trong số các biến không cơ sở có hệ số âm ( $G < 0$ ). Chọn biến cơ sở nhỏ nhất ( $x_1, x_2, x_3, w_1, w_2$ ).
- Chọn biến ra: y như đơn hình tính  $\frac{b_i}{a_{ij}} (= 0)$ .

$$\begin{array}{rcl} z' & = & -4x_1 \quad -5x_2 \\ \hline w_1 & = & 9 \quad -2x_1 \quad -2x_2 \\ w_2 & = & 4 \quad -x_1 \\ w_3 & = & 3 \quad \quad -x_2 \end{array}$$

$$\begin{array}{rcl} z' & = & -16 \quad +4w_2 \quad -5x_2 \\ \hline w_1 & = & 1 \quad +2w_2 \quad -2x_2 \\ x_1 & = & 4 \quad -w_2 \\ w_3 & = & 3 \quad \quad -x_2 \end{array}$$

$$\begin{array}{rcl} z' & = & -\frac{37}{2} \quad -w_2 \quad +\frac{5}{2}w_1 \\ \hline x_2 & = & \frac{1}{2} \quad +w_2 \quad -\frac{1}{2}w_1 \\ x_1 & = & 4 \quad -w_2 \\ w_3 & = & \frac{5}{2} \quad -w_2 \quad +\frac{1}{2}w_1 \end{array}$$

$$\begin{array}{rcl} z' & = & -21 \quad +w_3 \quad +2w_1 \\ \hline x_2 & = & 3 \quad -w_3 \\ x_1 & = & \frac{3}{2} \quad -w_3 \quad -\frac{1}{2}w_1 \\ w_3 & = & \frac{5}{2} \quad -w_3 \quad +\frac{1}{2}w_1 \end{array}$$

$$\text{Chọn } w_3 = w_1 = 0 \Rightarrow \begin{cases} x_2 = 3, & x_1 = \frac{3}{2} \\ w_2 = \frac{5}{2} \end{cases}$$

Vậy bài toán (P) có nghiệm tối ưu là  $x_1 = \frac{3}{2}$ ,  $x_2 = 3$  và giá trị tối ưu là  $z = 21$ .

### 1.3 Giải bài toán quy hoạch tuyến tính bằng phương pháp xoay bland

- Chọn biến vào: trong số các biến không cơ sở có hệ số âm ( $G < 0$ ). Chọn biến cơ sở nhỏ nhất ( $x_1, x_2, x_3, w_1, w_2$ ).
- Chọn biến ra: giống như phương pháp đơn hình:  $\frac{b_i}{a_{ij}} (= 0)$ .

Ví dụ:

$$\begin{array}{rcl} z' & = & -4x_1 \quad -5x_2 \\ \hline w_1 & = & 9 \quad -2x_1 \quad -2x_2 \\ w_2 & = & 4 \quad -x_1 \\ w_3 & = & 3 \quad \quad -x_2 \end{array}$$

$$\begin{array}{rcl} z' & = & -16 \quad +4w_2 \quad -5x_2 \\ \hline w_1 & = & 1 \quad +2w_2 \quad -2x_2 \\ x_1 & = & 4 \quad -w_2 \\ w_3 & = & 3 \quad \quad -x_2 \end{array}$$

$$\begin{array}{rcl} z' & = & -\frac{37}{2} - w_2 + \frac{5}{2}w_1 \\ \hline x_2 & = & \frac{1}{2} + w_2 - \frac{1}{2}w_1 \\ x_1 & = & 4 - w_2 \\ w_3 & = & \frac{5}{2} - w_2 + \frac{1}{2}w_1 \end{array}$$

$$\begin{array}{rcl} z' & = & -21 + w_3 + 2w_1 \\ \hline x_2 & = & 3 - w_3 \\ x_1 & = & \frac{3}{2} - w_3 - \frac{1}{2}w_1 \\ w_3 & = & \frac{5}{2} - w_3 + \frac{1}{2}w_1 \end{array}$$

$$\text{Chọn } w_3 = w_1 = 0 \Rightarrow \begin{cases} x_2 = 3, & x_1 = \frac{3}{2} \\ w_2 = \frac{5}{2} \end{cases}$$

Vậy bài toán (P) có nghiệm tối ưu là  $x_1 = \frac{3}{2}$ ,  $x_2 = 3$  và giá trị tối ưu là  $z = 21$ .

## 1.4 Giải bài toán quy hoạch tuyến tính bằng phương pháp đơn hình hai pha

Dùng bài toán không có nghiệm khởi tạo khả thi (ví dụ: có ràng buộc = với  $b < 0$ ).

$$(P) \longrightarrow (P') : \begin{cases} \min c^T x, \\ Ax \leq b \quad (\exists b_i > 0), \\ x \geq 0. \end{cases}$$

### • Pha 1:

1. Xây dựng bài toán bổ trợ:

$$(\Delta) : \begin{cases} \min x_0 \\ a_1^T x - x_0 \leq b_1, \\ a_2^T x - x_0 \leq b_2, \\ \vdots \\ x \geq 0, \\ x_0 \geq 0 \end{cases}$$

2. Lập từ vệt xuất pháp ( $\Delta$ ).

3. Biến vào:

Biến ra:  $w_i$  ứng với hệ số âm nhất  $\longrightarrow$  xoay  $\longrightarrow$  xuất hiện từ vệt mới (từ vệt chấp nhận được  $b_i > 0$  hoặc  $b_i = 0$ ).

4. Dùng thuật toán xoay đơn hình hoặc xoay bland.

### • Pha 2:

1. Lập từ vệtng chấp nhận được của  $(P')$  từ  $(P)$  và  $(\Delta)$ .

$$\begin{cases} z' = c^T x \\ (RB_\Delta) \text{ khi cho } x_0 = 0 \end{cases}$$

2. Xoay đơn hình rồi đưa ra kết quả .

**Ví dụ:** Xét bài toán

$$(P) : \begin{cases} \max & 2x_1 - 6x_2 \\ & -x_1 - x_2 - x_3 \leq -2 \\ & 2x_1 - x_2 + x_3 \leq 1 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_3 \geq 0 \end{cases}$$

Đưa về dạng chuẩn:

$$(P') : \begin{cases} -\min & -2x_1 + 6x_2 \\ & -x_1 - x_2 - x_3 \leq -2 \\ & 2x_1 - x_2 + x_3 \leq 1 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_3 \geq 0 \end{cases}$$

• **Pha 1:**

|                                                                                                                                                                                                                                             |                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{array}{rcl} \xi = & & x_0 \\ \hline w_1 = & -2 & +x_1 +x_2 +x_3 +x_0 \\ w_2 = & 1 & -2x_1 +x_2 -x_3 +x_0 \end{array}$                                                                                                               | $\begin{array}{rcl} \xi = & 2 & -x_1 -x_3 -x_3 +w_1 \\ \hline x_0 = & 2 & -x_1 -x_2 -x_3 +w_1 \\ w_2 = & 3 & -3x_1 -3x_3 +w_1 \end{array}$                                               |
| $\begin{array}{rcl} \xi = & 1 & +\frac{1}{3}w_2 -x_2 -\frac{1}{3}x_3 +\frac{2}{3}w_1 \\ \hline x_0 = & 2 & +\frac{1}{3}w_2 -x_2 -\frac{1}{3}x_3 +\frac{2}{3}w_1 \\ w_2 = & 1 & -\frac{1}{3}x_1 -\frac{2}{3}x_3 +\frac{1}{3}w_1 \end{array}$ | $\begin{array}{rcl} \xi = & & x_0 \\ \hline x_0 = & 1 & +\frac{1}{3}w_2 -x_0 -\frac{1}{3}x_3 +\frac{2}{3}w_1 \\ w_2 = & 1 & -\frac{1}{3}x_1 -\frac{2}{3}x_3 +\frac{1}{3}w_1 \end{array}$ |

• **Pha 2:**

|                                                                                                                                                                                                                                   |                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{array}{rcl} z' = & 4 & +\frac{8}{3}w_2 -\frac{2}{3}x_3 +\frac{10}{3}w_1 \\ \hline x_2 = & 1 & +\frac{1}{3}w_2 -\frac{1}{3}x_3 +\frac{2}{3}w_1 \\ x_1 = & 1 & -\frac{1}{3}w_2 -\frac{2}{3}x_3 +\frac{1}{3}w_1 \end{array}$ | $\begin{array}{rcl} z' = & 3 & +3w_2 +x_1 +3w_1 \\ \hline x_2 = & \frac{1}{2} & +\frac{1}{2}w_2 +\frac{1}{2}x_1 +\frac{1}{2}w_1 \\ x_1 = & \frac{3}{2} & -\frac{1}{3}w_2 -\frac{3}{2}x_1 +\frac{1}{2}w_1 \end{array}$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

$$\text{Chọn } w_2 = w_1 = x_1 = 0 \Rightarrow \begin{cases} x_2 = \frac{1}{2} \\ x_3 = \frac{3}{2} \end{cases}.$$

Vậy: bài toán có nghiệm tối ưu là:  $x_1 = 0, x_2 = \frac{1}{2}, x_3 = \frac{3}{2}$  và giá trị tối ưu là  $z = -3$ .

## 1.5 Xây dựng bài toán đối ngẫu (dual problem) cho bài toán gốc (primal) của bài toán QHTT

| $\min c^T x$                  | $\max y^T b$                  |
|-------------------------------|-------------------------------|
| $a_i^T x = b_i, i \in M_1$    | $y_i$ tự do, $i \in M_1$      |
| $a_i^T x \leq b_i, i \in M_2$ | $y_j \leq 0, i \in M_2$       |
| $a_i^T x \geq b_i, i \in M_3$ | $y_i \geq 0, i \in M_3$       |
| $x_J \geq 0, J \in N_1$       | $y^T A_J \leq c_J, J \in N_1$ |
| $x_J \leq 0, J \in N_2$       | $y^T A_J \geq c_J, J \in N_2$ |
| $x_J$ tự do, $J \in N_3$      | $y^T A_J = c_J, J \in N_3$    |

**Ví dụ:** Xét bài toán

$$(P) : \begin{cases} \min & -7x_2 + 3x_3 \\ & -x_1 - 2x_2 - x_3 \leq -3 \\ & -x_1 \leq 1 \\ & -x_3 \leq 0 \\ & x_1 + 3x_2 + 2x_3 = 27 \\ & 2x_1 - x_2 + 3x_3 = 12 \end{cases}$$

Bài toán đối ngẫu có dạng là:

$$(D) : \begin{cases} \max & -3y_1 + y_2 + 27y_3 + 12y_4 \\ & -y_1 - y_2 + y_3 + 2y_4 = 0 \\ & -2y_1 + 3y_3 - y_4 = -7 \\ & -y_1 + 2y_3 + 3y_4 \leq 3 \\ & y_1 \leq 0 \\ & y_2 \leq 0 \end{cases}$$

## 1.6 Giải bài toán quy hoạch tuyến tính bằng phương pháp đơn hình đối ngẫu

$$(P) \longrightarrow (P') : \begin{cases} \min c^T x \\ Ax \leq b \\ x \geq 0 \end{cases}$$

- **Bước 1:** Xây dựng từ vệtng xuất phát

$$\begin{array}{c} z = c_1x_1 + c_2x_2 + \dots + c_nx_n \\ \hline w_1 = b_1 + a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \\ w_2 = b_2 + a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \\ \vdots \\ w_n = b_n + a_{n1}x_1 + a_{n2}x_2 + \dots + a_{nn}x_n \end{array}$$

- **Bước 2:** Chọn biến vào/ra

1. Chọn biến ra: bị âm nhất (dòng  $j$ ).

2. Chọn biến vào: xét tỉ lệ  $\min \left\{ \frac{G_j}{a_{i_0j}} \right\} \longrightarrow$  cột  $J_0$ .

– **Bước 1:**  $\exists J_0 \longrightarrow$  xoay bình thường  $\longrightarrow$  từ vệtng ( $b_i \geq 0$ )  $\longrightarrow$  Đơn hình (gốc).

– **Bước 2:**  $\nexists J_0 \longrightarrow$  pha không áp dụng được  $\longrightarrow$  qua thuật toán đơn hình hai pha (đối ngẫu gốc),

**Ví dụ:** Xét bài toán

$$(P) : \begin{cases} \max & 2x_1 - 6x_2 \\ & -x_1 - x_2 - x_3 \leq -2 \\ & 2x_1 - x_2 + x_3 \leq 1 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_3 \geq 0 \end{cases}$$

Đưa  $(P)$  về dạng chuẩn:

$$(P') : \begin{cases} -\min & -2x_1 + 6x_2 \\ & -x_1 - x_2 - x_3 \leq -2 \\ & 2x_1 - x_2 + x_3 \leq 1 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_3 \geq 0 \end{cases}$$

Lập từ vịnh xuất phát

$$\begin{array}{rcl}
 z' = & -2x_1 & +6x_2 \\
 \hline
 w_1 = & -2 & +x_1 +x_2 +x_3 \\
 w_2 = & 1 & -2x_1 +x_2 -x_3 \\
 \hline
 z' = & 4 & +\frac{8}{3}w_2 +\frac{10}{3}w_1 -\frac{2}{3}x_3 \\
 x_2 = & 1 & +\frac{1}{3}x_1 +\frac{2}{3}w_1 -\frac{1}{3}x_3 \\
 x_1 = & 1 & -\frac{1}{3}x_1 +\frac{1}{3}w_1 -\frac{2}{3}x_3
 \end{array}
 \qquad
 \begin{array}{rcl}
 z' = & 12 & -8x_1 +6w_1 -6x_3 \\
 \hline
 x_2 = & 2 & -x_1 +w_1 -x_3 \\
 w_2 = & 3 & -3x_1 +w_1 -2x_3 \\
 \hline
 z' = & 3 & +3w_2 +3w_1 +x_1 \\
 x_2 = & \frac{1}{2} & +\frac{1}{2}w_2 +\frac{1}{2}w_1 +\frac{1}{2}x_1 \\
 x_2 = & \frac{3}{2} & -\frac{1}{2}w_2 +\frac{1}{2}w_1 -\frac{3}{2}x_1
 \end{array}$$

Chọn  $w_2 = w_1 = x_1 = 0 \Rightarrow \begin{cases} x_2 = \frac{1}{2} \\ x_3 = \frac{3}{2} \end{cases}$ .

Vậy; nghiệm tối ưu của bài toán (P) là:  $x_1 = 0, x_2 = \frac{1}{2}, x_3 = \frac{3}{2}$  và giá trị tối ưu là  $z = -3$ .

## 1.7 Giải bài toán quy hoạch tuyến tính bằng phương pháp đơn hình hai pha (đối ngẫu - gốc)

- **Pha 1:** Thiết lập bài toán bổ trợ:

$$(D) \begin{cases} \min & |c_1|x_1 + |c_2|x_2 + \dots + |c_n|x_n \\ & Ax \leq b \\ & x \geq 0 \end{cases}$$

Dùng thuật toán đơn hình đối ngẫu (mục 1.6) từ vịnh tối ưu của  $(P_{BT})$ .

- **Pha 2:** Dùng hàm mục tiêu:  $(P) : z = c^T x$  và ràng buộc của từ vịnh cuối phần 1  $\rightarrow$  từ vịnh chấp nhận được của bài toán gốc.

Dùng đơn hình gốc để suy ra nghiệm và giá trị tối ưu của bài toán gốc.

**Ví dụ:** Xét bài toán

$$(P) \begin{cases} \max & -2x_1 - 6x_2 \\ & -x_1 - x_2 - x_3 \leq -2 \\ & 2x_1 - x_2 + x_3 \leq 1 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_3 \geq 0 \end{cases}$$

Dạng chính tắc ( $P'$ ) của ( $P$ ) có dạng là:

$$(P') : \begin{cases} -\min & 2x_1 + 6x_2 \\ & -x_1 - x_2 - x_3 \leq -2 \\ & 2x_1 - x_2 + x_3 \leq 1 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_3 \geq 0 \end{cases}$$

• Pha 1:

$$\begin{array}{rcl} z'' = & 2x_1 & +6x_2 \\ \hline w_1 = & -2 & +x_1 +x_2 +x_3 \\ w_2 = & 1 & -2x_1 +x_2 -x_3 \end{array} \quad \begin{array}{rcl} z'' = & 4 & +2w_1 +4x_2 -2x_3 \\ \hline x_1 = & 2 & +w_1 -x_2 -x_3 \\ w_2 = & -2 & -2w_1 +3x_2 +x_3 \end{array}$$
  

$$\begin{array}{rcl} z'' = & 8 & +\frac{14}{3}w_1 +\frac{4}{3}w_2 -\frac{10}{3}x_3 \\ \hline x_1 = & 1 & +\frac{1}{3}w_1 -\frac{1}{3}w_2 -\frac{2}{3}x_3 \\ x_2 = & 1 & +\frac{2}{3}w_1 +\frac{1}{3}w_2 -\frac{1}{3}x_3 \end{array}$$

• Pha 2:

$$\begin{array}{rcl} z'' = & 3 & +3w_1 +3w_2 +5x_1 \\ \hline x_3 = & \frac{3}{2} & +\frac{1}{2}w_1 -\frac{1}{2}w_2 -\frac{3}{2}x_1 \\ x_2 = & \frac{1}{2} & +\frac{1}{2}w_1 +\frac{1}{2}w_2 +\frac{1}{2}x_1 \end{array} \quad \begin{array}{rcl} z' = & 3 & +3w_1 +3w_2 +x_1 \\ \hline x_3 = & \frac{3}{2} & +\frac{1}{2}w_1 -\frac{1}{2}w_2 -\frac{3}{2}x_1 \\ x_2 = & \frac{1}{2} & +\frac{1}{2}w_1 +\frac{1}{2}w_2 +\frac{1}{2}x_1 \end{array}$$

$$\text{Chọn } x_1 = w_1 = w_2 = 0 \Rightarrow \begin{cases} x_2 = \frac{1}{2} \\ x_3 = \frac{3}{2} \end{cases}$$

Vậy: nghiệm tối ưu của bài toán là  $x_1 = 0, x_2 = \frac{1}{2}, x_3 = \frac{3}{2}$  và giá trị tối ưu là  $z = -3$ .

## 2 Hướng dẫn code

### 2.1 Hướng dẫn code bài toán QHTT bằng phương pháp hình học

Ví dụ: Dùng phương pháp hình học giải bài toán quy hoạch tuyến tính sau

$$\begin{aligned} \min z &= -x_1 + x_2 \\ (P) : \begin{cases} -x_1 - 2x_2 \leq 6 \\ x_1 - 2x_2 \leq 4 \\ -x_1 + x_2 \leq 1 \\ x_1 \leq 0 \\ x_2 \leq 0 \end{cases} \end{aligned}$$

Code

```
1  import numpy as np
2
3  # 1) Định nghĩa các đường biên: a*x + b*y = c
4  #   -x1 - 2x2 = 6
5  #   x1 - 2x2 = 4
6  #   -x1 + x2 = 1
7  #   x1 = 0
8  #   x2 = 0
9  lines = [
10     (-1, -2, 6), # -x1 - 2x2 = 6
11     (1, -2, 4), # x1 - 2x2 = 4
12     (-1, 1, 1), # -x1 + x2 = 1
13     (1, 0, 0), # x1 = 0
14     (0, 1, 0) # x2 = 0
15 ]
16
17 # 2) Tìm giao điểm của mọi cặp đường biên
18 vertices = []
19 for i in range(len(lines)):
20     for j in range(i + 1, len(lines)):
21         a1, b1, c1 = lines[i]
22         a2, b2, c2 = lines[j]
23         A = np.array([[a1, b1],
24                        [a2, b2]])
25         B = np.array([c1, c2])
26         det = np.linalg.det(A)
27         if abs(det) < 1e-8:
28             # Hai đường gần như song song hoặc trùng => bỏ qua
29             continue
30         x, y = np.linalg.solve(A, B)
31         vertices.append((x, y))
32
33 # 3) Lọc ra các đỉnh thỏa mãn mọi bất đẳng thức
34 feasible = []
```



```

35 for x, y in vertices:
36     if (
37         -x - 2*y <= 6 + 1e-8 and # -x1 - 2x2 <= 6
38         x - 2*y <= 4 + 1e-8 and # x1 - 2x2 <= 4
39         -x + y <= 1 + 1e-8 and # -x1 + x2 <= 1
40         x <= 0 + 1e-8 and # x1 <= 0
41         y <= 0 + 1e-8 # x2 <= 0
42     ):
43         feasible.append((x, y))
44
45 # Loại bỏ trùng lặp do làm tròn
46 feasible = list(set((round(x, 8), round(y, 8)) for x, y in feasible))
47
48 # 4) Tính giá trị mục tiêu z = -x + y tại mỗi đỉnh khả thi
49 results = []
50 for x, y in feasible:
51     z = -x + y
52     results.append((x, y, z))
53
54 # 5) Tìm đỉnh có z nhỏ nhất
55 opt = min(results, key=lambda t: t[2])
56
57 # 6) In kết quả
58 print("Các đỉnh khả thi và giá trị z = -x + y:")
59 for x, y, z in sorted(results):
60     print(f" x1 = {x: .6f}, x2 = {y: .6f}, z = {z: .6f}")
61
62 x_opt, y_opt, z_opt = opt
63 print("\n\nNghiem tối ưu:")
64 print(f" x1* = {x_opt:.6f}, x2* = {y_opt:.6f}, z* = {z_opt:.6f}")
65

```

## Kết quả chạy thử

```

Các đỉnh khả thi và giá trị z = -x + y:
  x1 = -2.666667, x2 = -1.666667, z =  1.000000
  x1 = -1.000000, x2 = -2.500000, z = -1.500000
  x1 = -1.000000, x2 =  0.000000, z =  1.000000
  x1 =  0.000000, x2 = -2.000000, z = -2.000000
  x1 =  0.000000, x2 =  0.000000, z =  0.000000

Nghiem tối ưu:
  x1* = 0.000000, x2* = -2.000000, z* = -2.000000

```

## 2.2 Hướng dẫn code bài toán QHTT bằng phương pháp đơn hình

$$(P)' \left\{ \begin{array}{l} \min \quad y_1 - y_2 \\ y_1 + 2y_2 \leq 6 \\ -y_1 + 2y_2 \leq 4 \\ y_1 - y_2 \leq 1 \\ y_1 \geq 0 \\ y_2 \geq 0 \end{array} \right.$$

$$\begin{array}{rcl} z' & = & y_1 \quad -y_2 \\ \hline w_1 & = & 6 \quad -y_1 \quad -2y_2 \\ w_2 & = & 4 \quad +y_1 \quad -2y_2 \\ . & & \end{array} \quad \begin{array}{rcl} z' & = & -2 \quad +\frac{1}{2}y_1 \quad +\frac{1}{2}w_2 \\ \hline w_1 & = & 2 \quad -2y_1 \quad +w_2 \\ y_2 & = & 2 \quad +\frac{1}{2}y_1 \quad -\frac{1}{2}w_2 \\ w_3 & = & 3 \quad -\frac{1}{2}y_1 \quad -\frac{1}{2}w_2 \end{array}$$

$$\text{Chọn } y_1 = w_2 = 0 \Rightarrow \left\{ \begin{array}{l} w_1 = 2 \\ y_2 = 2 \\ w_3 = 3 \end{array} \right.$$

Vậy bài toán  $(P)$  có nghiệm tối ưu là  $x_1 = 0$ ,  $x_2 = -2$  và giá trị tối ưu là  $z = -2$ .

**Code**

```

1  """
2  Chương trình giải quy hoạch tuyến tính bằng Simplex (bảng cơ),
3  và tính thêm nghiệm "thực" xt dựa vào i_vec gốc (old_i).
4
5  Quy trình:
6  1) hambiendoi: chuẩn hóa bài toán gốc (min/max, biến ≥0/≤0/tự do) → min c^T x, A x ≤ b, x ≥ 0.
7  2) donhinh: giải bài toán đã chuẩn hóa, in từng bước bảng cơ, rồi tính nghiệm "thực" xt:
8      - x: nghiệm của bài toán đã chuẩn
9      - xt: nghiệm quy về biến gốc qua rule:
10         • Nếu old_i[u] = 1 → x_gốc = x_trans[u_ptr]; u_ptr += 1
11         • Nếu old_i[u] = -1 → x_gốc = -x_trans[u_ptr]; u_ptr += 1
12         • Nếu old_i[u] = 0 → x_gốc = x_trans[u_ptr] - x_trans[u_ptr+1]; u_ptr += 2
13
14
15  def hambiendoi(problem_type, num_vars, num_constraints, c, t, i_vec, mt, b):
16      """
17      Chuẩn hóa bài toán:
18      - Nếu problem_type = 1 (max), đổi dấu c để thành min.
19      - Xử lý biến i_vec[x]:
20          + i_vec[x] = 1 → x ≥ 0
21          + i_vec[x] = -1 → x ≤ 0 (y = -x ≥ 0)
22          + i_vec[x] = 0 → x tự do (x = x+ - x-, x+, x- ≥ 0)
23      - Xử lý ràng buộc t[r]:
24          + t[r] = 1 → "≥" → nhân -1 để thành "≤"
25          + t[r] = -1 → "≤" giữ nguyên
26          + t[r] = 0 → "=" → tách thành 2 ràng buộc "≤"
27
28      Trả về:
29      old_problem_type, old_t, old_i,
30      new_problem_type, new_t, new_i,
31      c_transformed, mt_transformed, b_transformed
32      """
33
34      old_problem_type = problem_type
35      old_t = t.copy()
36      old_i = i_vec.copy()
37
38      # 1) Nếu max, đổi dấu c để thành min
39      if problem_type == 1:
40          c = [-ci for ci in c]
41          new_problem_type = -1
42
43      # 2) Xử lý biến theo i_vec
44      original_n_vars = num_vars
45      num_rows = len(mt)
46
47      mt_transformed = [[] for _ in range(num_rows)]
48      c_transformed = []
49      new_i = []
50
51      for x in range(original_n_vars):
52          cx = c[x]
53          col = [mt[r][x] for r in range(num_rows)]
54          if i_vec[x] == 1:
55              # x ≥ 0: giữ nguyên
56              c_transformed.append(cx)
57              new_i.append(1)
58              for r in range(num_rows):
59                  mt_transformed[r].append(col[r])
60          elif i_vec[x] == -1:
61              # x ≤ 0: y = -x ≥ 0
62              c_transformed.append(-cx)
63              new_i.append(1)
64              for r in range(num_rows):
65                  mt_transformed[r].append(-col[r])

```

```

65         elif i_vec[x] == 0:
66             # x tự do: tách thành x+, x-
67             # x+:
68             c_transformed.append(cx)
69             new_i.append(1)
70             for r in range(num_rows):
71                 mt_transformed[r].append(col[r])
72             # x-:
73             c_transformed.append(-cx)
74             new_i.append(1)
75             for r in range(num_rows):
76                 mt_transformed[r].append(-col[r])
77         else:
78             raise ValueError(f"i_vec[{x}] bắt buộc là -1, 0 hoặc 1.")
79
80     # 3) Xử lý ràng buộc t[] trên mt_transformed
81     new_t = []
82     new_b = []
83     new_mt2 = []
84     for r in range(len(old_t)):
85         tr = old_t[r]
86         row = mt_transformed[r]
87         br = b[r]
88         if tr == 1:
89             #  $\geq \rightarrow$  nhân  $-1 \rightarrow \leq$ 
90             new_row = [-val for val in row]
91             new_bval = -br
92             new_t.append(-1)
93             new_mt2.append(new_row)
94
95             new_mt2.append(new_row)
96             new_b.append(new_bval)
97         elif tr == -1:
98             #  $\leq$  giữ nguyên
99             new_t.append(-1)
100             new_mt2.append(row.copy())
101             new_b.append(br)
102         elif tr == 0:
103             #  $= \rightarrow$  tách thành  $2 \leq$ 
104             new_t.append(-1)
105             new_mt2.append(row.copy())
106             new_b.append(br)
107             new_t.append(-1)
108             new_mt2.append([-val for val in row])
109             new_b.append(br)
110         else:
111             raise ValueError(f"t[{r}] bắt buộc là -1, 0 hoặc 1.")
112
113     return (
114         old_problem_type,
115         old_t,
116         old_i,
117         new_problem_type,
118         new_t,
119         new_i,
120         c_transformed,
121         new_mt2,
122         new_b

```

```

124 def donhinh(c, A, b, old_i, old_problem_type):
125     """
126     Thực hiện Simplex dạng bảng cơ và tính nghiệm thực xt:
127     - c: list độ dài n (hệ số mục tiêu, đã là 'min')
128     - A: list of lists m x n
129     - b: list độ dài m
130     - old_i: vector i_vec ban đầu (độ dài original_n_vars)
131
132     Trả về dict:
133     {
134     "objective": giá trị tối ưu,
135     "x": nghiệm biến chuẩn (list length = len(c)),
136     "xt": nghiệm quy về biến gốc (list length = len(old_i)),
137     "status": "optimal" / "unbounded"
138     }
139     """
140     m = len(A)
141     if m == 0:
142         # Nếu không có ràng buộc, nghiệm x = 0
143         x0 = [0.0]*len(c)
144         # Tính xt dựa trên old_i:
145         xt0 = []
146         ptr = 0
147         for idx, iv in enumerate(old_i):
148             if iv == 1:
149                 xt0.append(x0[ptr])
150                 ptr += 1
151             elif iv == -1:
152                 xt0.append(-x0[ptr])
153                 ptr += 1
154             else: # iv == 0
155                 xt0.append(x0[ptr] - x0[ptr+1])
156                 ptr += 2
157         ptr += 2
158         return {"objective": 0.0, "x": x0, "xt": xt0, "status": "optimal"}
159
160     n = len(A[0])
161     N = n + m # tổng số cột (biến gốc + slack)
162
163     # Tạo tên biến: x1..xn, s1..sm, cột RHS
164     col_names = [f"x{j+1}" for j in range(n)] + [f"s{i+1}" for i in range(m)] + ["RHS"]
165
166     # Khởi tạo tableau (m+1) x (N+1) với 0.0
167     tableau = [[0.0]*(N+1) for _ in range(m+1)]
168
169     # Ghi ràng buộc vào bảng (hàng 1..m)
170     for i in range(1, m+1):
171         for j in range(n):
172             tableau[i][j] = float(A[i-1][j])
173         tableau[i][n + (i-1)] = 1.0 # cột slack
174         tableau[i][N] = float(b[i-1]) # RHS
175
176     # Ghi hàng objective (i=0): cột 0..n-1 = c[j], slack & RHS = 0
177     for j in range(n):
178         tableau[0][j] = float(c[j])
179
180     # Biến cơ sở ban đầu: hàng i có slack i
181     basic_vars = [n + i for i in range(m)]
182
183     # Hàm phụ in tableau (bảng cơ)
184     def print_tableau(tab, basic_vars):
185         header = ["BV"] + col_names

```

```

183     def print_tableau(tab, basic_vars):
184         header = ["BV"] + col_names
185         print(" ".join(f"{h:>6}" for h in header))
186         for i in range(m+1):
187             if i == 0:
188                 bv = "z"
189             else:
190                 bv = col_names[basic_vars[i-1]]
191             row = [bv.rjust(6)]
192             for j in range(N+1):
193                 row.append(f"{tab[i][j]:>6.2f}")
194             print(" ".join(row))
195         print()
196
197     # In bảng khởi tạo
198     print("\n=== Bảng Simplex ban đầu ===")
199     print_tableau(tableau, basic_vars)
200
201     iteration = 0
202     while True:
203         iteration += 1
204         # (A) Tìm entering: cột j với tableau[0][j] < 0 nhiều nhất
205         entering = None
206         min_cost = 0.0
207         for j in range(N):
208             if tableau[0][j] < min_cost:
209                 min_cost = tableau[0][j]
210                 entering = j
211
212         if entering is None:
213             print(f"⇒ Đã tối ưu sau {iteration-1} bước.\n")
214             break
215
216         # (B) Tính ratios cho các hàng có hệ số dương ở cột entering
217         ratios = [float('inf')] * (m+1)
218         for i in range(1, m+1):
219             aij = tableau[i][entering]
220             if aij > 0:
221                 ratios[i] = tableau[i][N] / aij
222
223         # Nếu tất cả aij ≤ 0 → unbounded
224         if all(tableau[i][entering] ≤ 0 for i in range(1, m+1)):
225             print(f"⇒ Bài toán UNBOUNDED (cột {col_names[entering]} không có hệ số > 0).\n")
226             return {"objective": None, "x": None, "xt": None, "status": "unbounded"}
227
228         # (C) Tìm leaving: hàng i với ratios[i] nhỏ nhất
229         leaving = None
230         min_ratio = float('inf')
231         for i in range(1, m+1):
232             if ratios[i] < min_ratio - 1e-12:
233                 min_ratio = ratios[i]
234                 leaving = i
235
236         old_basic = basic_vars[leaving-1]
237
238         print(f"--- Bước {iteration} ---")
239         print(f"Biến vào   (entering): {col_names[entering]} (cost = {tableau[0][entering]:.2f})")
240         print(f"Biến ra     (leaving): {col_names[old_basic]} ở Row {leaving}, ratio = {min_ratio:.2f}\n")
241
242         # (D) Pivot tại (leaving, entering)
243         pivot_val = tableau[leaving][entering]

```

```

242     # (D) Pivot tại (leaving, entering)
243     pivot_val = tableau[leaving][entering]
244     # 1) Chia hàng leaving để pivot = 1
245     for j in range(N+1):
246         tableau[leaving][j] /= pivot_val
247     # 2) Loại cột entering ở các hàng khác
248     for i in range(m+1):
249         if i == leaving:
250             continue
251         factor = tableau[i][entering]
252         if abs(factor) > 1e-12:
253             for j in range(N+1):
254                 tableau[i][j] -= factor * tableau[leaving][j]
255
256     # (E) Cập nhật biến cơ sở
257     basic_vars[leaving-1] = entering
258
259     # In bảng sau pivot
260     print(f"--- Bảng sau pivot {iteration} ---")
261     print_tableau(tableau, basic_vars)
262
263     # Khi tối ưu, đọc nghiệm biến "chuẩn" x (cột j < n)
264     x = [0.0]*n
265     for i in range(1, m+1):
266         idx = basic_vars[i-1]
267         if idx < n:
268             val = tableau[i][N]
269             x[idx] = 0.0 if abs(val) < 1e-9 else val
270
271     # Giá trị objective = tableau[0][N]
272     obj = tableau[0][N]*old_problem_type
273
274     obj = tableau[0][N]*old_problem_type
275     if abs(obj) < 1e-9:
276         obj = 0.0
277
278     # 4) Tính nghiệm thực xt dựa vào old_i
279     xt = []
280     ptr = 0
281     for u, iv in enumerate(old_i):
282         if iv == 1:
283             # x_gốc = x_trans[ptr]
284             xt.append(x[ptr])
285             ptr += 1
286         elif iv == -1:
287             #  $x \leq 0 \rightarrow y = -x_{\text{trans}}[ptr]$ , nên x_gốc = -y
288             xt.append(-x[ptr])
289             ptr += 1
290         else: # iv == 0 (biến tự do)
291             #  $x = x^+ - x^-$ 
292             val = x[ptr] - x[ptr+1]
293             xt.append(val)
294             ptr += 2
295
296     return {"objective": obj, "x": x, "xt": xt, "status": "optimal"}
297
298 if __name__ == "__main__":
299     print("=== Giải quy hoạch tuyến tính bằng Simplex (bảng cơ) ===\n")
300
301     # 1) Nhập số biến và số ràng buộc
302     num_vars = int(input("Nhập số biến ban đầu (num_vars) = "))
303     num_constraints = int(input("Nhập số ràng buộc ban đầu (num_constraints) = "))

```

```

300 # 1) Nhập số biến và số ràng buộc
301 num_vars = int(input("Nhập số biến ban đầu (num_vars) = "))
302 num_constraints = int(input("Nhập số ràng buộc ban đầu (num_constraints) = "))
303
304 # 2) Nhập loại bài toán và vector c
305 problem_type = int(input("Nhập loại bài toán (1 = max, -1 = min) = "))
306 print(f"Nhập vector c (gồm {num_vars} số, cách nhau dấu cách):")
307 c0 = [float(x) for x in input().strip().split()]
308 if len(c0) != num_vars:
309     raise ValueError("Số phần tử c không khớp num_vars.")
310
311 # 3) Nhập vector t (ràng buộc)
312 print(f"Nhập vector t (gồm {num_constraints} số, mỗi phần tử ∈ {-1,0,1}, cách nhau dấu cách):")
313 t0 = [int(x) for x in input().strip().split()]
314 if len(t0) != num_constraints or any(v not in (-1,0,1) for v in t0):
315     raise ValueError("Vector t phải đúng độ dài và mỗi phần tử ∈ {-1,0,1}.")
316
317 # 4) Nhập vector i_vec (kiểu biến)
318 print(f"Nhập vector i_vec (gồm {num_vars} số, mỗi phần tử ∈ {-1,0,1}):")
319 i_vec0 = [int(x) for x in input().strip().split()]
320 if len(i_vec0) != num_vars or any(v not in (-1,0,1) for v in i_vec0):
321     raise ValueError("Vector i_vec phải đúng độ dài và mỗi phần tử ∈ {-1,0,1}.")
322
323 # 5) Nhập ma trận mt
324 print(f"Nhập ma trận mt ({num_constraints} hàng, mỗi hàng {num_vars} số):")
325 mt0 = []
326 for i in range(num_constraints):
327     row = [float(x) for x in input(f" Hàng {i+1}: ").strip().split()]
328     if len(row) != num_vars:
329         raise ValueError(f"Hàng {i+1} phải có {num_vars} phần tử.")
330     mt0.append(row)
331
332 # 6) Nhập vector b
333 print(f"Nhập vector b (gồm {num_constraints} số, cách nhau dấu cách):")
334 b0 = [float(x) for x in input().strip().split()]
335 if len(b0) != num_constraints:
336     raise ValueError("Vector b phải đúng độ dài num_constraints.")
337
338 # 7) Gọi hambiendoi
339 (old_pt, old_t, old_i,
340 new_pt, new_t, new_i,
341 c_tr, mt_tr, b_tr) = hambiendoi(problem_type, num_vars, num_constraints, c0, t0, i_vec0, mt0, b0)
342
343 print("\n=== Sau khi hambiendoi (chuẩn hóa) ===")
344 print("old_problem_type =", old_pt)
345 print("old_t =", old_t)
346 print("old_i =", old_i)
347 print("new_problem_type =", new_pt)
348 print("new_t =", new_t)
349 print("new_i =", new_i)
350 print("c_transformed =", c_tr)
351 print("mt_transformed ==")
352 for row in mt_tr:
353     print(" ", row)
354 print("b_transformed =", b_tr)
355
356 # 8) Gọi donhinh, truyền old_i để tính nghiệm thực xt
357 print("\n\n=== Giải bằng SIMPLEX (bảng cơ) và tính nghiệm thực ===")
358 result = donhinh(c_tr, mt_tr, b_tr, old_i, old_pt)
359
360 print("\n--- KẾT QUẢ CUỐI CÙNG ---")
361 print("status =", result["status"])
362
363 if result["status"] == "optimal":
364     print(f"objective = {result['objective']:.6f}")
365     print("x (bảng cơ)=", "[" + ", ".join(f"{v:.6f}" for v in result["x"]) + "]")
366     print("xt (thực) =", "[" + ", ".join(f"{v:.6f}" for v in result["xt"]) + "]")
367 else:
368     print("Bài toán UNBOUNDED (không có nghiệm hữu hạn).")

```

Kết quả chạy thử:



```

=== Bảng Simplex ban đầu ===
  BV      x1      x2      s1      s2      s3      RHS
  z       1.00    -1.00    0.00    0.00    0.00    0.00
  s1       1.00     2.00    1.00    0.00    0.00    6.00
  s2      -1.00     2.00    0.00    1.00    0.00    4.00
  s3       1.00    -1.00    0.00    0.00    1.00    1.00

--- Bước 1 ---
Biến vào   (entering): x2 (cost = -1.00)
Biến ra    (leaving): s2 ở Row 2, ratio = 2.00

--- Bảng sau pivot 1 ---
  BV      x1      x2      s1      s2      s3      RHS
  z       0.50     0.00    0.00    0.50    0.00    2.00
  s1       2.00     0.00    1.00   -1.00    0.00    2.00
  x2      -0.50     1.00    0.00    0.50    0.00    2.00
  s3       0.50     0.00    0.00    0.50    1.00    3.00

⇒ Đã tối ưu sau 1 bước.

--- KẾT QUẢ CUỐI CÙNG ---
status      = optimal
objective    = -2.000000
x (bảng cơ) = [0.000000, 2.000000]
xt (thực)   = [-0.000000, -2.000000]

```

## 2.3 Hướng dẫn code bài toán QHTT bằng phương pháp Bland

Code:

```

1  """
2  Chương trình giải quy hoạch tuyến tính bằng Simplex (bảng cơ),
3  với hàm hambien doi để chuẩn hóa và hàm bland (dùng quy tắc Bland) để giải.
4
5  - hambien doi: giữ nguyên như trước để chuẩn hóa bài toán gốc.
6  - bland: tương tự donhinh nhưng chọn biến entering theo Bland's rule
7    (chọn chỉ số cột nhỏ nhất trong số những cột có tableau[0][j] < 0).
8
9  Cuối cùng tính nghiệm "thực" xt dựa vào old_i.
10
11  Sử dụng:
12    1) Nhập num_vars, num_constraints.
13    2) Nhập problem_type (1=max, -1=min) và vector c.
14    3) Nhập vector t, vector i_vec.
15    4) Nhập ma trận mt và vector b.
16    5) Gọi hambien doi, rồi gọi bland(..., old_i) để giải.
17  """
18
19  def hambien doi(problem_type, num_vars, num_constraints, c, t, i_vec, mt, b):
20      """
21      Hàm chuẩn hóa (giữ nguyên từ trước).
22      """
23      old_problem_type = problem_type
24      old_t = t.copy()
25      old_i = i_vec.copy()
26
27      # Nếu max → đổi dấu c để thành min
28      if problem_type == 1:
29          c = [-ci for ci in c]
30      new_problem_type = -1
31
32      original_n_vars = num_vars
33      num_rows = len(mt)
34
35      mt_transformed = [[] for _ in range(num_rows)]
36      c_transformed = []
37      new_i = []
38
39      for x in range(original_n_vars):
40          cx = c[x]
41          col = [mt[r][x] for r in range(num_rows)]
42          if i_vec[x] == 1:
43              #  $x \geq 0$ : giữ nguyên
44              c_transformed.append(cx)
45              new_i.append(1)
46              for r in range(num_rows):
47                  mt_transformed[r].append(col[r])
48          elif i_vec[x] == -1:
49              #  $x \leq 0 \rightarrow y = -x \geq 0$ 
50              c_transformed.append(-cx)
51              new_i.append(1)
52              for r in range(num_rows):
53                  mt_transformed[r].append(-col[r])
54          elif i_vec[x] == 0:
55              # x tự do: tách thành  $x^+$ ,  $x^-$ 
56              #  $x^+$ :
57              c_transformed.append(cx)
58              new_i.append(1)
59              for r in range(num_rows):
60                  mt_transformed[r].append(col[r])
61              #  $x^-$ :
62              c_transformed.append(-cx)
63              new_i.append(1)
64              for r in range(num_rows):
65                  mt_transformed[r].append(-col[r])

```

```

65         mt_transformed[r].append(-col[r])
66     else:
67         raise ValueError(f"i_vec[{x}] bắt buộc là -1, 0 hoặc 1.")
68
69     new_t = []
70     new_b = []
71     new_mt2 = []
72     for r in range(len(old_t)):
73         tr = old_t[r]
74         row = mt_transformed[r]
75         br = b[r]
76         if tr == 1:
77             #  $\geq \rightarrow$  nhân  $-1 \rightarrow \leq$ 
78             new_row = [-val for val in row]
79             new_bval = -br
80             new_t.append(-1)
81             new_mt2.append(new_row)
82             new_b.append(new_bval)
83         elif tr == -1:
84             #  $\leq$  giữ nguyên
85             new_t.append(-1)
86             new_mt2.append(row.copy())
87             new_b.append(br)
88         elif tr == 0:
89             #  $= \rightarrow$  tách thành  $2 \leq$ 
90             new_t.append(-1)
91             new_mt2.append(row.copy())
92             new_b.append(br)
93             new_t.append(-1)
94             new_mt2.append([-val for val in row])
95             new_b.append(br)
96     else:
97         raise ValueError(f"t[{r}] bắt buộc là -1, 0 hoặc 1.")
98
99     return (
100         old_problem_type,
101         old_t,
102         old_i,
103         new_problem_type,
104         new_t,
105         new_i,
106         c_transformed,
107         new_mt2,
108         new_b
109     )
110
111
112 def bland(c, A, b, old_i, old_problem_type):
113     """
114     Hàm Simplex (bảng cơ) sử dụng Bland's rule để chọn biến entering.
115
116     Tham số:
117     - c: list độ dài n (hệ số mục tiêu sau khi hambiendoi)
118     - A: list of lists m x n (ma trận ràng buộc sau hambiendoi)
119     - b: list độ dài m (vector RHS sau hambiendoi)
120     - old_i: vector i_vec ban đầu (để tính nghiệm thực xt)
121
122     Trả về dict:

```

```

123     {
124         "objective": giá trị tối ưu,
125         "x": nghiệm biến đã chuẩn (length n),
126         "xt": nghiệm thực quy về biến gốc (length len(old_i)),
127         "status": "optimal"/"unbounded"
128     }
129     """
130     m = len(A)
131     if m == 0:
132         # Không có ràng buộc
133         x0 = [0.0]*len(c)
134         xt0 = []
135         ptr = 0
136         for iv in old_i:
137             if iv == 1:
138                 xt0.append(x0[ptr]); ptr += 1
139             elif iv == -1:
140                 xt0.append(-x0[ptr]); ptr += 1
141             else: # iv == 0
142                 xt0.append(x0[ptr] - x0[ptr+1]); ptr += 2
143         return {"objective": 0.0, "x": x0, "xt": xt0, "status": "optimal"}
144
145     n = len(A[0])
146     N = n + m # số cột (biến gốc + slack)
147
148     # Tên cột: x1..xn, s1..sm, RHS
149     col_names = [f"x{j+1}" for j in range(n)] + [f"s{i+1}" for i in range(m)] + ["RHS"]
150
151     # Khởi tạo tableau (m+1)x(N+1) = 0.0
152     tableau = [[0.0]*(N+1) for _ in range(m+1)]
153
154     # Điền ràng buộc
155     for i in range(1, m+1):
156         for j in range(n):
157             tableau[i][j] = float(A[i-1][j])
158         tableau[i][n + (i-1)] = 1.0 # cột slack
159         tableau[i][N] = float(b[i-1]) # RHS
160
161     # Hàng objective
162     for j in range(n):
163         tableau[0][j] = float(c[j])
164
165     # Biến cơ sở ban đầu: hàng i có slack i
166     basic_vars = [n + i for i in range(m)]
167
168     def print_tableau(tab, basic_vars):
169         header = ["BV"] + col_names
170         print(" ".join(f"{h:>6}" for h in header))
171         for i in range(m+1):
172             if i == 0:
173                 bv = "z"
174             else:
175                 bv = col_names[basic_vars[i-1]]
176             row = [bv.rjust(6)]
177             for j in range(N+1):
178                 row.append(f"{tab[i][j]:>6.2f}")
179             print(" ".join(row))
180         print()
181
182     # In bảng khởi tạo
183     print("\n=== Bảng Simplex ban đầu ===")
184     print_tableau(tableau, basic_vars)
185

```

```

185
186 iteration = 0
187 while True:
188     iteration += 1
189     # (A) Bland's rule: chọn entering = chỉ số cột j nhỏ nhất sao cho tableau[0][j] < 0
190     entering = None
191     for j in range(N):
192         if tableau[0][j] < 0:
193             entering = j
194             break
195
196     if entering is None:
197         print(f"→ Đã tối ưu sau {iteration-1} bước.\n")
198         break
199
200     # (B) Tính ratios: chỉ xét hàng i có tableau[i][entering] > 0
201     ratios = [float('inf')] * (m+1)
202     for i in range(1, m+1):
203         aij = tableau[i][entering]
204         if aij > 0:
205             ratios[i] = tableau[i][N] / aij
206
207     # Nếu không có hàng nào aij > 0 → unbounded
208     if all(tableau[i][entering] <= 0 for i in range(1, m+1)):
209         print(f"→ Bài toán UNBOUNDED (cột {col_names[entering]} không có hệ số > 0).\n")
210         return {"objective": None, "x": None, "xt": None, "status": "unbounded"}
211
212     # (C) Chọn leaving: hàng i có ratios[i] nhỏ nhất
213     leaving = None
214     min_ratio = float('inf')
215     for i in range(1, m+1):
216         if ratios[i] < min_ratio - 1e-12:
217             min_ratio = ratios[i]
218             leaving = i
219
220     old_basic = basic_vars[leaving-1]
221
222     print(f"--- Bước {iteration} ---")
223     print(f"Biến vào   (entering): {col_names[entering]} (value = {tableau[0][entering]:.2f})")
224     print(f"Biến ra     (leaving): {col_names[old_basic]} ở Row {leaving},   ratio = {min_ratio:.2f}\n")
225
226     # (D) Pivot tại (leaving, entering)
227     pivot_val = tableau[leaving][entering]
228     # 1) Chia hàng leaving
229     for j in range(N+1):
230         tableau[leaving][j] /= pivot_val
231     # 2) Loại cột entering khỏi các hàng khác
232     for i in range(m+1):
233         if i == leaving:
234             continue
235         factor = tableau[i][entering]
236         if abs(factor) > 1e-12:
237             for j in range(N+1):
238                 tableau[i][j] -= factor * tableau[leaving][j]
239
240     # (E) Cập nhật basic_vars
241     basic_vars[leaving-1] = entering
242
243     # In bảng sau pivot
244     print(f"--- Bảng sau pivot {iteration} ---")

```

```

243     # In bảng sau pivot
244     print(f"--- Bảng sau pivot {iteration} ---")
245     print_tableau(tableau, basic_vars)
246
247     # Đọc nghiệm x (cột j < n)
248     x = [0.0]*n
249     for i in range(1, m+1):
250         idx = basic_vars[i-1]
251         if idx < n:
252             val = tableau[i][N]
253             x[idx] = 0.0 if abs(val) < 1e-9 else val
254
255     # Objective = tableau[0][N]
256     obj = tableau[0][N]*old_problem_type
257     if abs(obj) < 1e-9:
258         obj = 0.0
259
260     # Tính nghiệm thực xt dựa vào old_i
261     xt = []
262     ptr = 0
263     for iv in old_i:
264         if iv == 1:
265             xt.append(x[ptr]); ptr += 1
266         elif iv == -1:
267             xt.append(-x[ptr]); ptr += 1
268         else: # iv == 0
269             xt.append(x[ptr] - x[ptr+1]); ptr += 2
270
271     return {"objective": obj, "x": x, "xt": xt, "status": "optimal"}
272
273
274 if __name__ == "__main__":
275     print("=== Giải quy hoạch tuyến tính bằng Bland's rule (bảng cơ) ===\n")
276
277     # 1) Nhập số biến và ràng buộc
278     num_vars = int(input("Nhập số biến ban đầu (num_vars) = "))
279     num_constraints = int(input("Nhập số ràng buộc ban đầu (num_constraints) = "))
280
281     # 2) Nhập problem_type và vector c
282     problem_type = int(input("Nhập loại bài toán (1 = max, -1 = min) = "))
283     print(f"Nhập vector c (gồm {num_vars} số, cách nhau dấu cách):")
284     c0 = [float(x) for x in input().strip().split()]
285     if len(c0) != num_vars:
286         raise ValueError("Số phần tử c không khớp num_vars.")
287
288     # 3) Nhập vector t (ràng buộc)
289     print(f"Nhập vector t (gồm {num_constraints} số, mỗi phần tử ∈ {-1,0,1}):")
290     t0 = [int(x) for x in input().strip().split()]
291     if len(t0) != num_constraints or any(v not in (-1,0,1) for v in t0):
292         raise ValueError("Vector t phải đúng độ dài và mỗi phần tử ∈ {-1,0,1}.")
293
294     # 4) Nhập vector i_vec (kiểu biến)
295     print(f"Nhập vector i_vec (gồm {num_vars} số, mỗi phần tử ∈ {-1,0,1}):")
296     i_vec0 = [int(x) for x in input().strip().split()]
297     if len(i_vec0) != num_vars or any(v not in (-1,0,1) for v in i_vec0):
298         raise ValueError("Vector i_vec phải đúng độ dài và mỗi phần tử ∈ {-1,0,1}.")
299
300     # 5) Nhập ma trận mt
301     print(f"Nhập ma trận mt ({num_constraints} hàng, mỗi hàng {num_vars} số):")
302     mt0 = []
303     for i in range(num_constraints):
304         row = [float(x) for x in input(f" Hàng {i+1}: ").strip().split()]
305         if len(row) != num_vars:

```

```

306         raise ValueError(f"Hàng {i+1} phải có {num_vars} phần tử.")
307     mt0.append(row)
308
309     # 6) Nhập vector b
310     print(f"Nhập vector b (gồm {num_constraints} số, cách nhau dấu cách):")
311     b0 = [float(x) for x in input().strip().split()]
312     if len(b0) != num_constraints:
313         raise ValueError("Vector b phải đúng độ dài num_constraints.")
314
315     # 7) Chuẩn hóa via hambien doi
316     (old_pt, old_t, old_i,
317      new_pt, new_t, new_i,
318      c_tr, mt_tr, b_tr) = hambien doi(problem_type, num_vars, num_constraints, c0, t0, i_vec0, mt0, b0)
319
320     print("\n=== Sau khi hambien doi (chuẩn hóa) ===")
321     print("old_problem_type =", old_pt)
322     print("old_t          =", old_t)
323     print("old_i          =", old_i)
324     print("new_problem_type =", new_pt)
325     print("new_t          =", new_t)
326     print("new_i          =", new_i)
327     print("c_transformed   =", c_tr)
328     print("mt_transformed  =", mt_tr)
329     for row in mt_tr:
330         print("    ", row)
331     print("b_transformed   =", b_tr)
332
333     # 8) Giải bằng Bland's rule
334     print("\n\n=== Giải bằng Bland's rule (bảng cơ) ===")
335     result = bland(c_tr, mt_tr, b_tr, old_i, old_pt)
336
337     # 8) Giải bằng Bland's rule
338     print("\n\n=== Giải bằng Bland's rule (bảng cơ) ===")
339     result = bland(c_tr, mt_tr, b_tr, old_i, old_pt)
340
341     print("\n--- KẾT QUẢ CUỐI CÙNG ---")
342     print("status      =", result["status"])
343     if result["status"] == "optimal":
344         print(f"objective   = {result['objective']:.6f}")
345         print("x (bảng cơ)=", "[" + ", ".join(f"{v:.6f}" for v in result["x"]) + "]")
346         print("xt (thực)  =", "[" + ", ".join(f"{v:.6f}" for v in result["xt"]) + "]")
347     else:
348         print("Bài toán UNBOUNDED (không có nghiệm hữu hạn).")

```

Ta tạo ra hàm hambien doi để chuyển bài toán về dạng chuẩn tắc

Nhập:

Loại bài toán  $\min(-1)$  hay  $\max(1)$ .

- Vecto  $c$ : hệ số hàm cần tối ưu.
- Vecto  $t$ : hệ số các ràng buộc
  - 1: ràng buộc  $\leq$
  - 1: ràng buộc  $\geq$
  - 0: ràng buộc  $=$ .
- Vecto  $i$ : hệ số giá trị các biến
  - 1: biến  $\leq 0$
  - 1: biến  $\geq 0$
  - 0: biến tự do.

Ma trận  $mt$ : hma trận các hệ số của các ràng buộc.

$b$ : hệ số bên phải các ràng buộc

Ta xét bài toán:

- Tối đa hóa:

$$x_1 - x_2$$

- Với các ràng buộc:

$$x_1 + x_2 \leq 10$$

$$x_1 \geq 1$$

$$x_2 \geq 1$$

- Với các điều kiện biến:

$x_1, x_2$  là **biến tự do**

- **Loại bài toán:** -1 (do là bài toán *max*, ta chuyển thành *min*)

- **Vectơ hệ số hàm mục tiêu  $c$ :**  $[1, -1]$

- **Loại ràng buộc  $t$ :**

– -1 cho ràng buộc  $\leq$

– 1 cho ràng buộc  $\geq$

$\rightarrow t = [-1, 1, 1]$

- **Loại biến  $i$ :**

– 0 cho biến tự do

$\rightarrow i = [0, 0]$

- **Vectơ hệ số vế phải  $b$ :**  $[10, 1, 1]$ .

Ma trận:

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Do ở đây  $x_1, x_2$  là biến tự do và sẽ được thay bằng 4 biến  $x'_1, x''_1, x'_2, x''_2$  nên sau khi giải xong ta thực hiện thuật toán rút gọn để lấy nghiệm chính xác.

Nhập các vectơ và ma trận:



```

Nhập số biến ban đầu (num_vars) = 2
Nhập số ràng buộc ban đầu (num_constraints) = 3
Nhập loại bài toán (1 = max, -1 = min) = 1
Nhập vector c (gồm 2 số, cách nhau dấu cách):
1 -1
Nhập vector t (gồm 3 số, mỗi phần tử  $\in \{-1,0,1\}$ ):
-1 1 1
Nhập vector i_vec (gồm 2 số, mỗi phần tử  $\in \{-1,0,1\}$ ):
0 0
Nhập ma trận mt (3 hàng, mỗi hàng 2 số):
Hàng 1: 1 1
Hàng 2: 1 0
Hàng 3: 0 1
Nhập vector b (gồm 3 số, cách nhau dấu cách):
10 1 1

```

Đây là các vectơ và ma trận sau khi biến đổi.

```

=== Sau khi hambien doi (chuẩn hóa) ===
old_problem_type = 1
old_t             = [-1, 1, 1]
old_i             = [0, 0]
new_problem_type  = -1
new_t             = [-1, -1, -1]
new_i             = [1, 1, 1, 1]
c_transformed     = [-1.0, 1.0, 1.0, -1.0]
mt_transformed    =
    [1.0, -1.0, 1.0, -1.0]
    [-1.0, 1.0, -0.0, 0.0]
    [-0.0, 0.0, -1.0, 1.0]
b_transformed     = [10.0, -1.0, -1.0]

```

Ta có thể thấy được hàm biến đổi đã hoàn thành nhiệm vụ chuyển bài toán về dạng chính tắc:

- $\text{new\_t} = [-1, -1, -1]$  chứng tỏ toàn bộ ràng buộc bây giờ là  $\leq$ .
- $\text{new\_i} = [1, 1, 1, 1]$  chứng tỏ 2 biến  $x_1, x_2$  giờ đã chuyển thành 4 biến  $x'_1, x''_1, x'_2, x''_2$  và cả 4 biến này đều  $\geq 0$ .

**Kết quả chạy thử:**

```

--- KẾT QUẢ CUỐI CÙNG ---
status      = optimal
objective    = 8.000000
x (bảng cơ) = [9.000000, 0.000000, 0.000000, -1.000000]
xt (thực)    = [9.000000, 1.000000]

```

Ở đây ta thấy được 4 biến  $x'_1$ ,  $x''_1$ ,  $x'_2$ ,  $x''_2$  ở vecto  $x$ , sau khi dùng thuật toán chuyển đổi lại ta có vecto  $x$  cần tìm.

**Ví dụ 2 Bland:**

$$\left\{ \begin{array}{l} \max \quad 4x_1 - 5x_2 \\ 2x_1 + 2x_2 \leq 9 \\ x_1 \leq 4 \\ x_2 \leq 3 \\ x_1, x_2 \geq 0. \end{array} \right.$$

=== Bảng Simplex ban đầu ===

| BV | x1    | x2    | s1   | s2   | s3   | RHS  |
|----|-------|-------|------|------|------|------|
| z  | -4.00 | -5.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| s1 | 2.00  | 2.00  | 1.00 | 0.00 | 0.00 | 9.00 |
| s2 | 1.00  | 0.00  | 0.00 | 1.00 | 0.00 | 4.00 |
| s3 | 0.00  | 1.00  | 0.00 | 0.00 | 1.00 | 3.00 |

$$\begin{array}{lcl} z' & = & -4x_1 - 5x_2 \\ \hline w_1 & = & 9 - 2x_1 - 2x_2 \\ w_2 & = & 4 - x_1 \\ w_3 & = & 3 - x_2 \end{array}$$

--- Bước 1 ---

Biến vào (entering): x1 (value = -4.00)

Biến ra (leaving): s2 ở Row 2, ratio = 4.00

--- Bảng sau pivot 1 ---

| BV | x1   | x2    | s1   | s2    | s3   | RHS   |
|----|------|-------|------|-------|------|-------|
| z  | 0.00 | -5.00 | 0.00 | 4.00  | 0.00 | 16.00 |
| s1 | 0.00 | 2.00  | 1.00 | -2.00 | 0.00 | 1.00  |
| x1 | 1.00 | 0.00  | 0.00 | 1.00  | 0.00 | 4.00  |
| s3 | 0.00 | 1.00  | 0.00 | 0.00  | 1.00 | 3.00  |

$$\begin{array}{rcl}
 z' & = & -16 \quad +4w_2 \quad -5x_2 \\
 \hline
 w_1 & = & 1 \quad +2w_2 \quad -2x_2 \\
 x_1 & = & 4 \quad -w_2 \\
 w_3 & = & 3 \quad \quad -x_2
 \end{array}$$

--- Bước 3 ---

Biến vào (entering): s2 (value = -1.00)

Biến ra (leaving): s3 ở Row 3, ratio = 2.50

--- Bảng sau pivot 3 ---

| BV | x1   | x2   | s1    | s2   | s3    | RHS   |
|----|------|------|-------|------|-------|-------|
| z  | 0.00 | 0.00 | 2.00  | 0.00 | 1.00  | 21.00 |
| x2 | 0.00 | 1.00 | 0.00  | 0.00 | 1.00  | 3.00  |
| x1 | 1.00 | 0.00 | 0.50  | 0.00 | -1.00 | 1.50  |
| s2 | 0.00 | 0.00 | -0.50 | 1.00 | 1.00  | 2.50  |

$$\begin{array}{rcl}
 z' & = & -21 \quad +w_3 \quad +2w_1 \\
 \hline
 x_2 & = & 3 \quad -w_3 \\
 x_1 & = & \frac{3}{2} \quad -w_3 \quad -\frac{1}{2}w_1 \\
 w_3 & = & \frac{5}{2} \quad -w_3 \quad +\frac{1}{2}w_1
 \end{array}$$

Ta thấy rõ bảng khớp với giải tay ở từng bước

--- KẾT QUẢ CUỐI CÙNG ---

status = optimal

objective = 21.000000

x (bảng cơ) = [1.500000, 3.000000]

xt (thực) = [1.500000, 3.000000]

Kết quả cuối cùng: hoàn toàn chính xác.

## 2.4 Hướng dẫn code bài toán QHTT bằng thuật toán hai pha

```
1  import numpy as np
2  import pandas as pd
3
4  def two_phase_simplex_min(A, b, c):
5      """
6      Giải bài toán LP (min) dưới dạng:
7          min z = c^T x
8          st. A x ≤ b, x ≥ 0,
9      với khả năng b_i < 0. Mọi ràng buộc ban đầu đều hiểu là "≤".
10     Thuật toán sử dụng Two-Phase Simplex.
11     Mỗi khi pivot, in ra tableau hiện tại cùng biến vào (entering) và biến ra (leaving).
12
13     Đầu vào:
14     - A: ma trận numpy kích thước (m x n)
15     - b: numpy array độ dài m (có thể chứa phần tử âm)
16     - c: numpy array độ dài n (hệ số của hàm mục tiêu minimization)
17
18     Trả về:
19     - solution: dict chứa giá trị x1..xn tối ưu
20     - optimal_value: giá trị tối thiểu của c^T x
21     """
22
23     m, n = A.shape
24
25     # --- Bước 0: Khởi tạo constraint_senses = "≤" cho mọi ràng buộc ---
26     constraint_senses = ["≤"] * m
27
28     # --- Bước 1: Xử lý b_i < 0: nhân cả hàng với -1, đổi ≤ thành ≥ ---
29     A_proc = A.astype(float).copy()
30     b_proc = b.astype(float).copy()
31     for i in range(m):
32         if b_proc[i] < 0:
33             A_proc[i, :] *= -1
34             b_proc[i] *= -1
35             constraint_senses[i] = "≥"
36
37     # --- Bước 2: Xây dựng A_ext (thêm slack/surplus và artificial) ---
38     A_ext = A_proc.copy()
39     var_names = [f"x{j+1}" for j in range(n)] # tên x1..xn
40     slack_names = []
41     art_names = []
42     basis = []
43
44     num_slack = 0
45     num_art = 0
46
47     for i, sense in enumerate(constraint_senses):
48         if sense == "≤":
49             # thêm biến slack s_i ≥ 0
50             slack_col = np.zeros(m)
51             slack_col[i] = 1
52             A_ext = np.hstack((A_ext, slack_col.reshape(-1,1)))
53             s_name = f"s{num_slack+1}"
54             var_names.append(s_name)
55             slack_names.append(s_name)
56             basis.append(s_name)
57             num_slack += 1
58
59         elif sense == "≥":
60             # thêm biến surplus (-s_i) và artificial
61             surp_col = np.zeros(m)
```

```

61     surp_col = np.zeros(m)
62     surp_col[i] = -1
63     A_ext = np.hstack((A_ext, surp_col.reshape(-1,1)))
64     s_name = f"s{num_slack+1}"
65     var_names.append(s_name)
66     slack_names.append(s_name)
67     num_slack += 1
68
69     art_col = np.zeros(m)
70     art_col[i] = 1
71     A_ext = np.hstack((A_ext, art_col.reshape(-1,1)))
72     a_name = f"a{num_art+1}"
73     var_names.append(a_name)
74     art_names.append(a_name)
75     basis.append(a_name)
76     num_art += 1
77
78     else:
79         raise ValueError("Chỉ hỗ trợ ràng buộc '<=' hoặc '>=' sau khi tiền xử lý.")
80
81     total_vars = A_ext.shape[1]
82
83     # --- Phase I: xây dựng hàm mục tiêu loại bỏ tất cả artificial ---
84     c_phase1 = np.zeros(total_vars)
85     for a in art_names:
86         idx = var_names.index(a)
87         c_phase1[idx] = -1.0
88
89     tableau = np.zeros((m+1, total_vars+1))
90     tableau[:m, :total_vars] = A_ext
91
92     tableau[:m, :total_vars] = A_ext
93     tableau[:m, -1] = b_proc
94     tableau[-1, :total_vars] = c_phase1
95
96     # Điều chỉnh hàng Obj Phase I
97     for a in art_names:
98         row = basis.index(a)
99         tableau[-1, :] += tableau[row, :]
100
101     def print_tableau(tbl, basis, entering=None, leaving=None):
102         """
103         In tableau dưới dạng DataFrame (pandas) với index = basis + ['Obj'].
104         Nếu entering/leaving không None, in dòng thông báo trước khi in bảng.
105         """
106         if entering is not None and leaving is not None:
107             print(f"Entering = {entering}, Leaving = {leaving}")
108             df = pd.DataFrame(tbl, columns=var_names + ["RHS"])
109             df.index = basis + ["Obj"]
110             print(df, "\n")
111
112     print("=== Phase I: Bảng Simplex ban đầu ===")
113     # Lần đầu in bảng Phase I, chưa có biến vào/ra ⇒ truyền None
114     print_tableau(tableau, basis, entering=None, leaving=None)
115
116     def pivot(tbl, basis):
117         """
118         Thực hiện 1 bước pivot trên tableau:
119         - Chọn biến vào (entering): cột có hệ số dương lớn nhất trong hàng Obj.
120         - Nếu không có hệ số dương => optimal.
121         - Chọn biến ra (leaving): min ratio (RHS / A_ij) với A_ij > 0.
122         - Pivot: chuẩn hóa và khử các hàng còn lại.
123         Trả về: (entering_name, leaving_name, is_optimal)

```

```

121     Trả về: (entering_name, leaving_name, is_optimal)
122     """
123     rows, cols = tbl.shape
124     num_vars = cols - 1 # trừ cột RHS
125
126     obj_row = tbl[-1, :num_vars]
127     entering_idx = np.argmax(obj_row)
128     if obj_row[entering_idx] <= 1e-9:
129         return None, None, True
130
131     ratios = np.full(rows-1, np.inf)
132     for i in range(rows-1):
133         a_ij = tbl[i, entering_idx]
134         if a_ij > 1e-9:
135             ratios[i] = tbl[i, -1] / a_ij
136
137     leaving_idx = np.argmin(ratios)
138     if ratios[leaving_idx] == np.inf:
139         raise ValueError("Bài toán không giới hạn (Unbounded) trong Phase I.")
140
141     pivot_val = tbl[leaving_idx, entering_idx]
142     tbl[leaving_idx, :] /= pivot_val
143     for i in range(rows):
144         if i != leaving_idx:
145             tbl[i, :] -= tbl[i, entering_idx] * tbl[leaving_idx, :]
146
147     entering_var = var_names[entering_idx]
148     leaving_var = basis[leaving_idx]
149     basis[leaving_idx] = entering_var
150
151     return entering_var, leaving_var, False
152
153 # Chạy Phase I
154 step = 0
155 while True:
156     step += 1
157     entering, leaving, is_opt = pivot(tableau, basis)
158     if is_opt:
159         print(f"=== Phase I đã tối ưu sau {step-1} bước pivot ===")
160         # Ở đây không có biến vào/ra nữa → in bảng cuối cùng nhưng không in thêm dòng "Entering/Leaving"
161         print_tableau(tableau, basis, entering=None, leaving=None)
162         break
163     print(f"--- Phase I, pivot {step} ---")
164     print_tableau(tableau, basis, entering, leaving)
165
166 # Kiểm tra khả thi
167 if abs(tableau[-1, -1]) > 1e-8:
168     print("Bài toán vô nghiệm (Infeasible) sau Phase I.")
169     return None, None
170
171 # --- Loại bỏ cột artificial ---
172 # Cần lưu vị trí cũ để loại bỏ đúng
173 for a in art_names:
174     idx = var_names.index(a)
175     tableau = np.delete(tableau, idx, axis=1)
176     var_names.pop(idx)
177 total_vars = len(var_names)
178
179 # Thay thế trong basis nếu còn artificial

```

```

179     # Thay thế trong basis nếu còn artificial
180     for i in range(m):
181         if basis[i] in art_names:
182             for j in range(total_vars):
183                 if abs(tableau[i, j]) > 1e-9:
184                     basis[i] = var_names[j]
185                     break
186
187     # --- Phase II: thiết lập hàm mục tiêu gốc (min => max -c) ---
188     c_phase2 = np.zeros(total_vars)
189     for j in range(n):
190         c_phase2[j] = -c[j]
191
192     tableau[-1, :total_vars] = c_phase2
193     tableau[-1, -1] = 0.0
194     for i in range(m):
195         var = basis[i]
196         idx_var = var_names.index(var)
197         coef = tableau[-1, idx_var]
198         if abs(coef) > 1e-12:
199             tableau[-1, :] -= coef * tableau[i, :]
200
201     print("=== Phase II: Bảng Simplex khởi tạo ===")
202     print_tableau(tableau, basis, entering=None, leaving=None)
203
204     # Chạy Phase II
205     step = 0
206     while True:
207         step += 1
208         entering, leaving, is_opt = pivot(tableau, basis)
209         if is_opt:
210             if is_opt:
211                 print(f"=== Phase II đã tối ưu sau {step-1} bước pivot ===")
212                 print_tableau(tableau, basis, entering=None, leaving=None)
213                 break
214             print(f"--- Phase II, pivot {step} ---")
215             print_tableau(tableau, basis, entering, leaving)
216
217     # --- Đọc nghiệm ---
218     x_vals = np.zeros(total_vars)
219     for i in range(m):
220         var = basis[i]
221         idx = var_names.index(var)
222         x_vals[idx] = tableau[i, -1]
223
224     solution = {f"x{j+1}": float(x_vals[j]) for j in range(n)}
225     optimal_value = -float(tableau[-1, -1]) # do chúng ta đã maximize -c^T x
226
227     return solution, optimal_value
228
229     # ===== Ví dụ minh họa =====
230     if __name__ == "__main__":
231
232
233         A = np.array([
234             [-1, -1, -1],
235             [2, -1, 1]
236         ], dtype=float)
237         b = np.array([-2, 1], dtype=float)
238         c = np.array([-2, 6, 0], dtype=float)

```

```

233     A = np.array([
234         [-1, -1, -1],
235         [ 2, -1,  1]
236     ], dtype=float)
237     b = np.array([-2, 1], dtype=float)
238     c = np.array([-2, 6, 0], dtype=float)
239
240     sol, val = two_phase_simplex_min(A, b, c)
241     print("Nghiem tối ưu:", sol)
242     print("Giá trị tối thiểu:", val)
243

```



## 2.5 Hướng dẫn code bài toán QHTT xây dựng bài toán đối ngẫu (dual problem) cho bài toán gốc (primal)

| $\min c^T x$                  | $\max y^T b$                  |
|-------------------------------|-------------------------------|
| $a_i^T x = b_i, i \in M_1$    | $y_i$ tự do, $i \in M_1$      |
| $a_i^T x \leq b_i, i \in M_2$ | $y_j \leq 0, i \in M_2$       |
| $a_i^T x \geq b_i, i \in M_3$ | $y_i \geq 0, i \in M_3$       |
| $x_J \geq 0, J \in N_1$       | $y^T A_J \leq c_J, J \in N_1$ |
| $x_J \leq 0, J \in N_2$       | $y^T A_J \geq c_J, J \in N_2$ |
| $x_J$ tự do, $J \in N_3$      | $y^T A_J = c_J, J \in N_3$    |

Chuyển đổi đối ngẫu:

- $c_{moi} = b$
- $b_{moi} = c$
- $t_{moi} = i$  (vectơ ràng buộc mới là vectơ biến cũ)
- $i_{moi} = t$  (vectơ biến mới là vectơ ràng buộc cũ)
- $mt_{moi} = mt.T$

Bài toán:

### (P) – Bài toán Primal

$$\min x_1 + 2x_2 - 3x_3$$

subject to:

$$x_1 + x_2 = 5$$

$$x_1 - x_2 + 2x_3 = 6$$

$$x_1 + x_2 - x_3 \leq -2$$

$$-3x_2 + 5x_3 \geq 6$$

biến (variables):

$$x_1 \geq 0$$

$$x_2 \leq 0$$

$$x_3: \text{tự do}$$

### (D) – Bài toán Dual

$$\max 5y_1 + 6y_2 - 2y_3 + 6y_4$$

subject to:

$$y_1 + y_2 + y_3 \leq 1 \quad (\text{RBD})$$

$$y_1 - y_2 + y_3 - 3y_4 \geq 2 \quad (\text{RB})$$

$$2y_2 - y_3 + 5y_4 = -3 \quad (\text{RB})$$

biến (variables):

$$y_1: \text{tự do}$$

$$y_2: \text{tự do}$$

$$y_3 \leq 0$$

$$y_4 \geq 0$$

Code:

```

1  import numpy as np
2
3  # Hàm nhập vector từ người dùng
4  def nhap_vector(ten):
5      v = input(f"Nhập các phần tử của vector {ten}, phân tách bằng dấu cách: ")
6      return np.array([float(x) for x in v.strip().split()])
7
8  # Hàm nhập ma trận từ người dùng
9  def nhap_ma_tran():
10     hang = int(input("Nhập số hàng của ma trận mt: "))
11     print("Nhập từng hàng, mỗi phần tử cách nhau bởi dấu cách:")
12     mat = []
13     for i in range(hang):
14         dong = input(f"Hàng {i+1}: ")
15         mat.append([float(x) for x in dong.strip().split()])
16     return np.array(mat)
17
18 # Nhập các vector
19 c = nhap_vector('c')
20 b = nhap_vector('b')
21 t = nhap_vector('t')
22 i = nhap_vector('i')
23
24 # Nhập ma trận
25 mt = nhap_ma_tran()
26
27 # Tạo các vector và ma trận mới
28 c_moi = b
29 b_moi = c
30 t_moi = i
31 i_moi = t
32 mt_moi = mt.T
33
34 # Xuất kết quả
35 print("c_moi =", c_moi)
36 print("b_moi =", b_moi)
37 print("t_moi =", t_moi)
38 print("i_moi =", i_moi)
39 print("mt_moi (chuyển vị của mt):")
40 print(mt_moi)
41
42

```

Kết quả chạy thử:

```

c_moi = [ 5.  6. -2.  6.]
b_moi = [ 1.  2. -3.]
t_moi = [ 1. -1.  0.]
i_moi = [ 0.  0. -1.  1.]
mt_moi (chuyển vị của mt):
[[ 1.  1.  1.  0.]
 [ 1. -1.  1. -3.]
 [ 0.  2. -1.  5.]]

```

## 2.6 Hướng dẫn code bài toán QHTT bằng phương pháp đơn hình đối ngẫu

$$(P) : \left\{ \begin{array}{l} \max \quad 2x_1 - 6x_2 \\ -x_1 - x_2 - x_3 \leq -2 \\ 2x_1 - x_2 + x_3 \leq 1 \\ x_1 \geq 0 \\ x_2 \geq 0 \\ x_3 \geq 0 \end{array} \right.$$

```

1  """
2
3      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
4      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
5      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
6      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
7      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
8      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
9      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
10     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
11     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
12     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
13     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
14     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
15     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
16     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
17     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
18     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
19     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
20     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
21     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
22     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
23     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
24     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
25     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
26     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
27     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
28     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
29     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
30     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
31     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
32     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
33     |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
34
35     """
36
37     Online Python Compiler.
38     Code, Compile, Run and Debug python program online.
39     Write your code in this editor and press "Run" button to execute it.
40
41     """
42
43     import numpy as np
44     import pandas as pd
45
46     def dual_simplex_min(A, b, c):
47         """
48         Giải bài toán LP (min) dưới dạng:
49             min    z = c^T x
50             st.    A x ≤ b,    x ≥ 0,
51         với A là ma trận (m x n), b có thể chứa phần tử âm, và c là vector dài n.
52         Thuật toán sử dụng Dual-Simplex (khi cần, tự động chuyển sang Primal Simplex nếu vẫn chưa tối ưu).
53         Mọi ràng buộc được hiểu là "≤", mọi biến x_j ≥ 0.
54         Mỗi khi thực hiện một bước pivot (Dual hoặc Primal), hàm sẽ in ra tableau hiện tại cùng
55         "Entering" và "Leaving".
56
57         Trả về:
58             - solution: dict chứa giá trị x1..xn tối ưu
59             - optimal_value: giá trị tối thiểu của c^T x
60         """
61
62         m, n = A.shape
63
64         # --- 1) Khởi tạo bảng bằng cách thêm biến slack s1..sm ---
65         # Không đổi dấu b; nếu b_i < 0, khi đó slack lúc đầu = b_i < 0 → primal infeasible, dùng Dual simplex.
66         A_ext = np.hstack((A.astype(float), np.eye(m, dtype=float)))
67         total_vars = n + m # x1..xn và s1..sm
68
69         # Danh sách tên biến: x1..xn, s1..sm
70         var_names = [f"x{j+1}" for j in range(n)] + [f"s{i+1}" for i in range(m)]
71         # Basis ban đầu: các biến slack s1..sm
72         basis = [f"s{i+1}" for i in range(m)]
73
74         # --- 2) Khởi tạo tableau ---
75         # Tableau kích thước (m+1) x (total_vars+1)
76         tableau = np.zeros((m+1, total_vars+1))
77         # Hàng ràng buộc: A_ext và RHS = b
78         tableau[:m, :total_vars] = A_ext
79         tableau[:m, -1] = b.astype(float)
80         # Hàng Obj: hệ số reduced cost r_j = c_j - c_B * (B^-1)N, nhưng ban đầu c_B = 0 (slack có cost 0),
81         # nên r_j = c_j cho j=0..n-1 (x), và r_j = 0 cho j=n..n+m-1 (slack).
82         tableau[-1, :n] = c.astype(float)
83         tableau[-1, n:] = 0.0
84         tableau[-1, -1] = 0.0 # RHS của Obj
85
86         def print_tableau(tbl, basis, entering=None, leaving=None):
87             """
88             In tableau dưới dạng DataFrame (pandas) với index = basis + ['Obj'].
89             Nếu entering và leaving không None, in thêm dòng thông báo trước khi in bảng.
90             """
91             if entering is not None and leaving is not None:
92                 print(f"Entering = {entering}, Leaving = {leaving}")
93             df = pd.DataFrame(tbl, columns=var_names + ["RHS"])
94             df.index = basis + ["Obj"]
95             print(df, "\n")
96
97         # In bảng khởi tạo (chưa pivot)
98         print("=== Bảng Simplex ban đầu ===")
99         print_tableau(tableau, basis, entering=None, leaving=None)

```

```

66
67 # --- 3) Định nghĩa hàm pivot_dual và pivot_primal ---
68 def pivot_dual(tbl, basis):
69     """
70     Thực hiện một bước Dual Simplex trong bảng tbl:
71     - Tìm hàng leaving_idx có RHS (b_i) âm nhất (nhỏ nhất).
72     Nếu không còn b_i < 0, trả is_opt=True.
73     - Ở hàng leaving_idx, tìm các cột j sao cho a_ij < 0.
74     Với mỗi ứng viên j, tính ratio_j = r_j / a_ij, trong đó r_j = tbl[-1,j].
75     Chọn entering_idx sao cho ratio_j nhỏ nhất.
76     Nếu không có cột nào a_ij < 0 → LP vô nghiệm.
77     - Thực hiện pivot tại (leaving_idx, entering_idx), cập nhật basis.
78     Trả về: (entering_var, leaving_var, is_optimal)
79     """
80     rows, cols = tbl.shape
81     num_vars = cols - 1
82
83     # 1) Tìm hàng leaving_idx: b_i âm nhất
84     rhs = tbl[:, -1]
85     leaving_idx = int(np.argmin(rhs))
86     if rhs[leaving_idx] >= -1e-9:
87         # Không còn b_i < 0 → dual feasible, kết thúc Dual phase
88         return None, None, True
89
90     # 2) Ở hàng leaving_idx, tìm cột với a_ij < 0
91     row_k = tbl[leaving_idx, :num_vars]
92     obj_row = tbl[-1, :num_vars]
93     candidates = []
94     for j in range(num_vars):
95         if row_k[j] < -1e-9:
96             ratio = obj_row[j] / row_k[j]
97
98             ratio = obj_row[j] / row_k[j]
99             candidates.append((ratio, j))
100
101     if not candidates:
102         raise ValueError("LP infeasible (Dual phase): không tìm được cột để pivot.")
103
104     # Chọn j sao cho ratio_j nhỏ nhất
105     _, entering_idx = min(candidates, key=lambda x: x[0])
106
107     # 3) Pivot tại (leaving_idx, entering_idx)
108     pivot_val = tbl[leaving_idx, entering_idx]
109     tbl[leaving_idx, :] /= pivot_val
110     for i in range(rows):
111         if i != leaving_idx:
112             tbl[i, :] -= tbl[i, entering_idx] * tbl[leaving_idx, :]
113
114     entering_var = var_names[entering_idx]
115     leaving_var = basis[leaving_idx]
116     basis[leaving_idx] = entering_var
117
118     return entering_var, leaving_var, False
119
120 def pivot_primal(tbl, basis):
121     """
122     Thực hiện một bước Primal Simplex trong bảng tbl (minimization):
123     - Tìm cột entering_idx sao cho r_j = tbl[-1,j] < 0 (reduced cost âm nhất).
124     Nếu không còn r_j < 0 → optimal (is_opt=True).
125     - Với entering_idx, tìm hàng i có a_ij > 0, tính ratio_i = b_i / a_ij và chọn
126     leaving_idx có ratio nhỏ nhất. Nếu không có a_ij > 0 → LP unbounded.

```

```

123         leaving_idx có ratio nhỏ nhất. Nếu không có  $a_{ij} > 0 \rightarrow$  LP unbounded.
124         - Pivot tại (leaving_idx, entering_idx), cập nhật basis.
125     Trả về: (entering_var, leaving_var, is_optimal)
126     """"
127     rows, cols = tbl.shape
128     num_vars = cols - 1
129
130     # 1) Tìm cột entering_idx:  $r_j$  âm nhất
131     obj_row = tbl[-1, :num_vars]
132     entering_idx = int(np.argmin(obj_row))
133     if obj_row[entering_idx] >= -1e-9:
134         # Không còn  $r_j < 0 \rightarrow$  tối ưu về mặt primal
135         return None, None, True
136
137     # 2) Tính ratio để chọn leaving row
138     ratios = np.full(m, np.inf)
139     for i in range(m):
140         a_ij = tbl[i, entering_idx]
141         if a_ij > 1e-9:
142             ratios[i] = tbl[i, -1] / a_ij
143     leaving_idx = int(np.argmin(ratios))
144     if ratios[leaving_idx] == np.inf:
145         raise ValueError("LP unbounded (Primal phase): không tìm được hàng để pivot.")
146
147     # 3) Pivot tại (leaving_idx, entering_idx)
148     pivot_val = tbl[leaving_idx, entering_idx]
149     tbl[leaving_idx, :] /= pivot_val
150     for i in range(rows):
151         if i != leaving_idx:
152             if i != leaving_idx:
153                 tbl[i, :] -= tbl[i, entering_idx] * tbl[leaving_idx, :]
154
155     entering_var = var_names[entering_idx]
156     leaving_var = basis[leaving_idx]
157     basis[leaving_idx] = entering_var
158
159     return entering_var, leaving_var, False
160
161 # --- 4) Vòng lặp chính: Dual Simplex (sửa  $b_i < 0$ ) rồi Primal (sửa  $r_j < 0$ ) ---
162 step_dual = 0
163 step_primal = 0
164
165 while True:
166     # 4.1) Nếu còn  $b_i < 0 \rightarrow$  Dual Simplex
167     rhs = tableau[:, -1]
168     if np.any(rhs < -1e-9):
169         step_dual += 1
170         entering, leaving, is_opt = pivot_dual(tableau, basis)
171         if is_opt:
172             # Trường hợp hiếm khi pivot_dual trả True ngay (nếu tất cả  $b_i \geq 0$ ).
173             print(f"=== Sau {step_dual-1} bước Dual Simplex: không còn  $b_i < 0$  ===")
174             print_tableau(tableau, basis, entering=None, leaving=None)
175             # Sau khi hết dual-infeasible, có thể vẫn còn  $r_j < 0$ , chuyển tiếp xuống phần Primal.
176         else:
177             print(f"--- Dual Simplex, pivot {step_dual}: ---")
178             print_tableau(tableau, basis, entering, leaving)
179             continue

```

```

180 # 4.2) Bây giờ mọi  $b_i \geq 0$ . Nếu còn  $r_j < 0 \rightarrow$  Primal Simplex
181 obj_row = tableau[-1, :total_vars]
182 if np.any(obj_row < -1e-9):
183     step_primal += 1
184     entering, leaving, is_opt = pivot_primal(tableau, basis)
185     if is_opt:
186         print(f"=== Sau {step_primal-1} bước Primal Simplex: không còn  $r_j < 0$  ===")
187         print_tableau(tableau, basis, entering=None, leaving=None)
188         break
189     else:
190         print(f"--- Primal Simplex, pivot {step_primal}: ---")
191         print_tableau(tableau, basis, entering, leaving)
192         continue
193
194 # Nếu không còn  $b_i < 0$  và không còn  $r_j < 0 \rightarrow$  tối ưu đồng thời primal & dual
195 print("=== Tối ưu: không còn  $b_i < 0$  và không còn  $r_j < 0$  ===")
196 print_tableau(tableau, basis, entering=None, leaving=None)
197 break
198
199 # --- 5) Đọc nghiệm  $x_1..x_n$  ---
200 x_vals = np.zeros(total_vars)
201 for i in range(m):
202     var = basis[i]
203     idx = var_names.index(var)
204     x_vals[idx] = tableau[i, -1]
205
206 solution = {f"x{j+1}": float(x_vals[j]) for j in range(n)}
207 # Giá trị tối thiểu  $c^T x = \text{tableau\_Obj\_RHS}$ , bởi vì ta lưu  $r_j = c_j$ , và dòng Obj có RHS =  $z$ 
208 optimal_value = float(np.dot(c, np.array([solution[f"x{j+1}"] for j in range(n)])))
209
210 return solution, optimal_value
211
212
213 # ===== Ví dụ minh họa =====
214
215 if __name__ == "__main__":
216     # Ví dụ LP (min):
217     # min  $z = 2x_1 + 3x_2 + x_3$ 
218     # st.
219     #  $x_1 - 2x_2 + 1x_3 \leq -1$ 
220     #  $2x_1 + 1x_2 + 3x_3 \leq 5$ 
221     #  $-1x_1 + 1x_2 + 2x_3 \leq 3$ 
222     #  $x_1, x_2, x_3 \geq 0$ 
223
224     A = np.array([
225         [-1, -1, -1],
226         [2, -1, 1]
227     ], dtype=float)
228     b = np.array([-2, 1], dtype=float)
229     c = np.array([-2, 6, 0], dtype=float)
230
231     sol, val = dual_simplex_min(A, b, c)
232     print("Nghiệm tối ưu:", sol)
233     print("Giá trị tối thiểu:", val)

```

```

=== Bảng Simplex ban đầu ===

```

|       | $x_1$ | $x_2$ | $x_3$ | $s_1$ | $s_2$ | RHS  |
|-------|-------|-------|-------|-------|-------|------|
| $s_1$ | -1.0  | -1.0  | -1.0  | 1.0   | 0.0   | -2.0 |
| $s_2$ | 2.0   | -1.0  | 1.0   | 0.0   | 1.0   | 1.0  |
| Obj   | -2.0  | 6.0   | 0.0   | 0.0   | 0.0   | 0.0  |

$$\begin{array}{rcl}
 z' = & -2x_1 & +6x_2 \\
 \hline
 w_1 = & -2 & +x_1 & +x_2 & +x_3 \\
 w_2 = & 1 & -2x_1 & +x_2 & -x_3
 \end{array}$$

```

--- Dual Simplex, pivot 1: ---
Entering = x2, Leaving = s1
      x1    x2    x3    s1    s2    RHS
x2    1.0    1.0    1.0   -1.0   -0.0    2.0
s2    3.0    0.0    2.0   -1.0    1.0    3.0
Obj   -8.0    0.0   -6.0    6.0    0.0   -12.0

```

$$\begin{array}{rcl}
 z' = & 12 & -8x_1 & +6w_1 & -6x_3 \\
 \hline
 x_2 = & 2 & -x_1 & +w_1 & -x_3 \\
 w_2 = & 3 & -3x_1 & +w_1 & -2x_3
 \end{array}$$

```

--- Primal Simplex, pivot 1: ---
Entering = x1, Leaving = s2
      x1    x2    x3    s1    s2    RHS
x2    0.0    1.0    0.333333 -0.666667 -0.333333  1.0
x1    1.0    0.0    0.666667 -0.333333  0.333333  1.0
Obj    0.0    0.0   -0.666667  3.333333  2.666667  -4.0

```

$$\begin{array}{rcl}
 z' = & 4 & +\frac{8}{3}w_2 & +\frac{10}{3}w_1 & -\frac{2}{3}x_3 \\
 \hline
 x_2 = & 1 & +\frac{1}{3}x_1 & +\frac{2}{3}w_1 & -\frac{1}{3}x_3 \\
 x_1 = & 1 & -\frac{1}{3}x_1 & +\frac{1}{3}w_1 & -\frac{2}{3}x_3
 \end{array}$$



```

--- Primal Simplex, pivot 2: ---
Entering = x3, Leaving = x1
      x1  x2  x3  s1  s2  RHS
x2 -0.5  1.0  0.0 -0.5 -0.5  0.5
x3  1.5  0.0  1.0 -0.5  0.5  1.5
Obj  1.0  0.0  0.0  3.0  3.0 -3.0

=== Tối ưu: không còn b_i < 0 và không còn r_j < 0 ===
      x1  x2  x3  s1  s2  RHS
x2 -0.5  1.0  0.0 -0.5 -0.5  0.5
x3  1.5  0.0  1.0 -0.5  0.5  1.5
Obj  1.0  0.0  0.0  3.0  3.0 -3.0

```

$$\begin{aligned}
 z' &= 3 + 3w_2 + 3w_1 + x_1 \\
 x_2 &= \frac{1}{2} + \frac{1}{2}w_2 + \frac{1}{2}w_1 + \frac{1}{2}x_1 \\
 x_3 &= \frac{3}{2} - \frac{1}{2}w_2 + \frac{1}{2}w_1 - \frac{3}{2}x_1
 \end{aligned}$$

Nghiệm tối ưu: {'x1': 0.0, 'x2': 0.5, 'x3': 1.5}  
 Giá trị tối thiểu: 3.0

Đáp án hoàn toàn chính xác.

## 2.7 Hướng dẫn code bài toán QHTT bằng phương pháp đơn hình hai pha (đôi ngẫu - gốc)

```

1  import numpy as np
2  import pandas as pd
3
4  def two_phase_dual_original_min(A, b, c):
5
6
7      m, n = A.shape
8
9      # === Phase 1: Build auxiliary LP tableau ===
10     # A_ext = [A | I_m] so that each constraint becomes A_i.x + s_i = b_i, with s_i >= 0.
11     A_ext = np.hstack((A.astype(float), np.eye(m, dtype=float)))
12     var_names = [f"x{j+1}" for j in range(n)] + [f"s{i+1}" for i in range(m)]
13     basis = [f"s{i+1}" for i in range(m)]
14     total_vars = n + m
15
16     # Construct initial tableau (m+1 rows x (total_vars+1) columns):
17     tableau = np.zeros((m+1, total_vars+1))
18     # Rows 0..m-1: (A_ext | b)
19     tableau[:m, :total_vars] = A_ext
20     tableau[:m, -1] = b.astype(float)
21     # Obj row (row m): coefficients = |c_j| for x_j, 0 for all slacks, RHS = 0
22     for j in range(n):
23         tableau[-1, j] = abs(c[j])
24
25     for i in range(m):
26         tableau[-1, :] -= tableau[i, :]
27
28     def print_tableau(tbl, basis, entering=None, leaving=None):
29         """Print the current tableau as a pandas DataFrame, labeling the basic variables."""
30         if entering is not None and leaving is not None:
31             print(f"Entering = {entering}, Leaving = {leaving}")
32         df = pd.DataFrame(tbl, columns=var_names + ["RHS"])
33         df.index = basis + ["Obj"]
34         print(df, "\n")
35
36     print("=== Phase 1 (Auxiliary) Initial Tableau ===")

```

```

36 print("=== Phase 1 (Auxiliary) Initial Tableau ===")
37 print_tableau(tableau, basis)
38
39 # --- Dual Simplex Pivot (Phase 1) ---
40 def dual_pivot(tbl, basis):
41     rows, cols = tbl.shape
42     num_vars = cols - 1
43
44     # 1) Pick the row with the most negative RHS
45     rhs = tbl[:, -1]
46     leaving_idx = int(np.argmin(rhs))
47     if rhs[leaving_idx] >= -1e-9:
48         # No negative RHS → dual phase is done
49         return None, None, True
50
51     # 2) In that row, pick any column j with a_{k,j} < 0
52     row_k = tbl[leaving_idx, :num_vars]
53     obj_row = tbl[-1, :num_vars]
54     candidates = []
55     for j in range(num_vars):
56         if row_k[j] < -1e-9:
57             ratio = obj_row[j] / row_k[j]
58             candidates.append((ratio, j))
59     if not candidates:
60         # Still a negative RHS but no a_{k,j} < 0 → infeasible
61         raise ValueError("Auxiliary LP is infeasible (Dual Simplex failed).")
62
63     # Choose the entering column j with minimal ratio
64     _, entering_idx = min(candidates, key=lambda x: x[0])
65
66     # 3) Pivot on (leaving_idx, entering_idx)
67     pivot_val = tbl[leaving_idx, entering_idx]
68     tbl[leaving_idx, :] /= pivot_val
69     for i in range(rows):
70         if i != leaving_idx:
71             if i != leaving_idx:
72                 tbl[i, :] -= tbl[i, entering_idx] * tbl[leaving_idx, :]
73
74     entering_var = var_names[entering_idx]
75     leaving_var = basis[leaving_idx]
76     basis[leaving_idx] = entering_var
77     return entering_var, leaving_var, False
78
79 # Run Dual Simplex until no RHS < 0 remains
80 step1 = 0
81 try:
82     while True:
83         step1 += 1
84         entering, leaving, done = dual_pivot(tableau, basis)
85         if done:
86             print(f"=== Phase 1 complete after {step1-1} pivots ===")
87             print_tableau(tableau, basis)
88             break
89         print(f"--- Phase 1, pivot {step1} ---")
90         print_tableau(tableau, basis, entering, leaving)
91 except ValueError:
92     print("Auxiliary LP is infeasible. Therefore the original LP has no feasible solution.")
93     return None, None
94
95 # If the auxiliary Obj-value w* > 0, original LP is infeasible
96 w_val = tableau[-1, -1]
97 if w_val > 1e-8:
98     print("Auxiliary LP optimal w > 0 → original LP infeasible.")
99     return None, None
100
101 tableau[-1, :] = 0.0
102 for j in range(n):
103     tableau[-1, j] = -c[j]

```

```

102 tableau[-1, :] = 0.0
103 for j in range(n):
104     tableau[-1, j] = -c[j]
105 # Now subtract each basic variable's multiple so that its reduced cost becomes 0
106 for i in range(m):
107     var = basis[i]
108     idx = var_names.index(var)
109     coef = tableau[-1, idx]
110     if abs(coef) > 1e-12:
111         tableau[-1, :] -= coef * tableau[i, :]
112
113 print("=== Phase 2 (Primal) Initial Tableau ===")
114 print_tableau(tableau, basis)
115
116 def primal_pivot(tbl, basis):
117
118     rows, cols = tbl.shape
119     num_vars = cols - 1
120
121     # 1) Find entering
122     obj_row = tbl[-1, :num_vars]
123     entering_idx = int(np.argmax(obj_row))
124     if obj_row[entering_idx] <= 1e-9:
125         return None, None, True # no positive reduced cost → optimal
126
127     # 2) Ratio test to pick leaving
128     ratios = np.full(m, np.inf)
129     for i in range(m):
130         a_ij = tbl[i, entering_idx]
131         if a_ij > 1e-9:
132             ratios[i] = tbl[i, -1] / a_ij
133     leaving_idx = int(np.argmin(ratios))
134     if ratios[leaving_idx] == np.inf:
135         raise ValueError("Original LP is unbounded during Phase 2.")
136
137     # 3) Pivot
138     pivot_val = tbl[leaving_idx, entering_idx]
139     tbl[leaving_idx, :] /= pivot_val
140     for i in range(rows):
141         if i != leaving_idx:
142             tbl[i, :] -= tbl[i, entering_idx] * tbl[leaving_idx, :]
143
144     entering_var = var_names[entering_idx]
145     leaving_var = basis[leaving_idx]
146     basis[leaving_idx] = entering_var
147     return entering_var, leaving_var, False
148
149 # Run Primal Simplex until no reduced cost > 0
150 step2 = 0
151 try:
152     while True:
153         step2 += 1
154         entering, leaving, done = primal_pivot(tableau, basis)
155         if done:
156             print(f"=== Phase 2 complete after {step2-1} pivots ===")
157             print_tableau(tableau, basis)
158             break
159         print(f"--- Phase 2, pivot {step2} ---")
160         print_tableau(tableau, basis, entering, leaving)
161 except ValueError:
162     print("Original LP is unbounded in Phase 2.")
163     return None, None
164
165 # === Read off x1..xn from the final tableau ===
166 x_vals = np.zeros(total_vars)
167 for i in range(m):
168     var = basis[i]
169     idx = var_names.index(var)

```

```

165     # === Read off x1..xn from the final tableau ===
166     x_vals = np.zeros(total_vars)
167     for i in range(m):
168         var = basis[i]
169         idx = var_names.index(var)
170         x_vals[idx] = tableau[i, -1]
171
172     solution = {f"x{j+1}": float(x_vals[j]) for j in range(n)}
173     optimal_value = float(np.dot(c, np.array([solution[f"x{j+1}"] for j in range(n)])))
174     return solution, optimal_value
175
176
177 if __name__ == "__main__":
178
179
180     A = np.array([
181         [-1, -1, -1],
182         [2, -1, 1]
183     ], dtype=float)
184     b = np.array([-2, 1], dtype=float)
185     c = np.array([2, 6, 0], dtype=float)
186
187     sol, val = two_phase_dual_original_min(A, b, c)
188     print("Optimal solution:", sol)
189     print("Optimal value:", val)
190

```